Codex nad legacy kódem

Produktový den týmu Money ERP

Ing. Jan Polišenský

OpenAI Codex a agentní vývoj, 2. den



Ing. Jan Polišenský

CTO Lakmoos AI | AI Researcher | University Lecturer

O mně

- Výzkumník AI na VUT FIT v Brně
- Specializace: neurosymbolická AI, LLM, detekce hrozeb
- Aktivní přednášející na univerzitách (např. Oxford)
- Publikace, akademický výzkum, vedení diplomových prací

Lakmoos AI (CTO & co-founder)

- Vedu tým ~30 inženýrů
- 3× Slush Top 50 inovativních startupů
- Finalista Nápad roku 2024
- Coding agenty používáme každý den ve vlastním vývoji



jan@lakmoos.com

Vzorový legacy repozitář na dnešek

Mzdy a DPH: kus legacy ERP v malém, se všemi pastmi

```
# .NET 10 SDK (bez admina: viz README)
winget install Microsoft.DotNet.SDK.10

git clone https://github.com/poli-cz/erp-legacy-codex-kurz
cd erp-legacy-codex-kurz
dotnet test     # sami, ať proběhne restore

# mezi pokusy: vlastní větev nebo
git stash -u
# úplně od začátku (smaže i poznámky)
git restore . ; git clean -fd
```

github.com/poli-cz/erp-legacy-codex-kurz

Vymyšlená firma a data, žádný firemní kód: smíte ho dát jakémukoli agentovi.

DNES

Program dne

Sedm bloků. Pořadí 4 a 5 podle toho, co vás bude pálit.

1	**Rentgen vašeho setupu**	šest vrstev, config, sandbox, hooky, skilly	60 min
2	**Kontext pro miliony řádků**	hierarchie AGENTS.md, mapy, kódování	50 min
3	**Legacy vzory s agentem**	seams, charakterizace, paralelní běh, expand–contract	100 min
4	**Money ERP: redesign a Azure DevOps**	WinForms, DevExpress, testovací nástroj, PR	60 min
5	**Ověřování, pipeline a náklady**	žebřík ověření, review v CI, kredity	50 min
6	**Pravidla týmu a plán**	kdo co vlastní, co měřit, 30/60/90 dní	30 min
7	**Více agentů najednou**	sub-agenti, protokoly, předávka, zásady	40 min

Obsahu je víc, než stihneme. Kde se zasekneme na vašem problému, tam zůstaneme.

Kde dnes jste?

Jedno písmeno, podle toho upravím poměr diskuze a cvičení

A

Chat vedle IDE

Kopíruju kód do ChatGPT nebo Codexu a zpět.

B

Agent v repu

Codex CLI nebo rozšíření, malé úlohy.

C

Agent s pravidly

Máme AGENTS.md, skilly nebo vlastní postup.

D

Agent v procesu

Review, testy nebo CI s agentem.

Většina A a B: víc bloku 1 a 2. Hodně C a D: víc bloku 3 a 5.

BLOK 1 Z 7

Rentgen vašeho setupu

01

Šest vrstev, kde se rozhoduje o kvalitě i o kreditech

60 MINUT

Postup dne

Šest vrstev setupu

Kdo nastavuje co, kde to leží a co se typicky pokazí

Vrstva	Co nastavuje	Kde a kdo	Typická chyba
1 Plán a admin	povolené modely, kredity, limity skupin, vynucená pravidla	Admin Console, requirements.toml / IT	jen upozornění na útratu, žádný strop přečerpání
2 Klient	CLI, Windows app, VS Code, JetBrains; verze	notebook / každý	stará verze, jiné chování než kolega
3 Osobní config	model, effort, profily, sandbox, prostředí	~/.codex/*.config.toml / každý	drahý model nastálo, tokeny v env
4 Instrukce repa	AGENTS.md, skilly, hooky, projektový config	repo / tým přes PR	wiki zkopírovaná do AGENTS.md
5 Integrace	MCP: Azure DevOps (cloud), SQL, dokumentace	config / tým + IT	zápisová práva „pro jistotu“
6 Pipeline	build, testy, review agentem, brány	Azure Pipelines / tým	agent může, CI nekontroluje

Vrstvy 1 a 6 jsou firemní, 3 je osobní, 4 a 5 jsou týmové. Dnes hlavně 3 až 6.

Diagnostika za pět minut

Spusťte u sebe. Co se liší od souseda, je téma diskuze.

Terminál (PowerShell)

```
› cmd /c ver  # 26100 = 24H2, 26200 = 25H2
› codex --version
  # aktuální stabilní 0.162.1 (9. 10.)
› codex login status
› codex features list
› codex mcp list
› type $env:USERPROFILE\.codex\config.toml
› dir $env:USERPROFILE\.codex\*.config.toml
› dir $env:USERPROFILE\.agents\skills

# co agent opravdu dostane jako vstup:
› codex debug prompt-input
```

Uvnitř Codexu (TUI)

```
/status          model, effort, okno, tokeny
/usage           spotřeba účtu, reset limitu
/debug-config    vrstvy configu a jejich zdroj
/permissions     sandbox a schvalování
/model           přepnout model a effort
/skills          které skilly agent vidí
/hooks           hooky a jejich důvěra
/mcp verbose     servery a nástroje
```

/debug-config ukáže, odkud se která hodnota vzala. Konec dohadů „u mě to funguje“.

Co se v Codexu změnilo a může vám rozbít setup

Stav k 13. 10. 2026. Pokud máte config z jara, něco z toho máte.

1	**GPT-5.5 skončil 14. 10.**	config nebo spravovaný config s pevně nastaveným gpt-5.5 dnes nefunguje	*přepsat na aktuální model*
2	**approval_policy**	„untrusted“ zrušeno, config s ním může zablokovat start; „on-failure“ se tiše mění na on-request	*přepsat na on-request*
3	**Profily v2**	[profiles.x] + -p x končí chybou; profil je soubor ~/.codex/x.config.toml	*přesunout do souborů*
4	**--full-auto**	neexistuje od 0.147; náhrada --sandbox workspace-write nebo --approve-for-me	*opravit skripty a návody*
5	**codex mcp-server**	odstraněn v 0.154; náhrada app-server (JSON-RPC, není MCP)	*jen pokud jste ho používali*
6	**Dokumentace**	developers.openai.com/codex přesměrovává na learn.chatgpt.com/docs	*aktualizovat odkazy ve wiki*

Verze klienta je součást setupu. Tým s pěti verzemi má pět různých nástrojů.

Kam tečou kredity: sazby modelů v Codexu

Kredity za 1 milion tokenů, standardní úroveň. Poměry jsou důležitější než čísla.

Model	Vstup	Cache	Výstup	Na co
GPT-6 Astra	250	25	1 250	výjimečně: návrh, těžká chyba (5× Sol)
GPT-6.1 Sol (výchozí)	50	2,5	250	běžná práce, effort low až medium
GPT-6 Sol	50	5	250	předchozí verze, cache 2× dražší
GPT-6 Luna	2,5	0,25	12,5	průzkum, testy, přejmenování (20× levnější)
Fast (/fast)				2× spotřeba na kreditech, jen když čekáte

Výstup 5× dražší než vstup, cache 10–20× levnější. Přepnutí modelu uprostřed vlákna zahodí cache.

Kde Codex bere nastavení

Pět vrstev. Při startu je Codex složí dohromady.



- Vyšší vrstva přebíjí nižší.
- Každý soubor obsahuje jen to, co mění. Zbytek zdědí.
- Bez configu běží výchozí hodnoty Codexu.
- Odkud se vzala která hodnota: /debug-config.

Config nastavuje, requirements.toml hlídá. Uživatel si config změní, requirements ne.

Pět míst s nastavením

Kde soubor leží, kdo ho píše a kdy platí

Vrstva	Kde leží (Windows)	Kdo ho píše	Kdy platí
Osobní config	%USERPROFILE%\.codex\config.toml	každý sám	vždy, je to základ
Profil	%USERPROFILE%\.codex\levny.config.toml	každý sám	jen s codex -p levny
Týmový config	<repo>\.codex\config.toml (+ .codex\rules)	tým, přes PR	jen v důvěryhodném (trusted) projektu
Přepínače	codex -m gpt-6-luna -c web_search="disabled"	každý při spuštění	jen to jedno spuštění
Requirements	%ProgramData%\OpenAI\Codex\ requirements.toml nebo cloud	IT, admin	vždy, uživatel nepřebije

Co má platit pro všechny, patří do repa. Co si každý ladí sám, do osobního configu.

Co se do configu píše

Deset klíčů, které stačí znát. Zbytek nechte výchozí.

Klíč	Co nastavuje	Doporučení
model	který model pracuje	gpt-6.1-sol; dražší jen v profilu
model_reasoning_effort	jak dlouho model přemýšlí (low až ultra)	medium; high jen v profilu
sandbox_mode	co smí agent na disku (viz sandbox)	workspace-write
approval_policy	kdy se agent ptá na schválení	on-request
web_search	hledání na webu	cached (výchozí) nebo disabled
[sandbox_workspace_write]	síť a další zapisovatelné složky	network_access = false
[shell_environment_policy]	které proměnné prostředí agent vidí	odfiltrovat hesla a tokeny
[agents]	model a počet pomocných agentů	levný model, max. 3 naráz
[windows] sandbox	čím se na Windows postaví sandbox	mxc nebo elevated
project_root_markers	podle čeho Codex pozná kořen repa	repo bez Gitu: [".git", "Hlavni.sln"]

Nejdražší chyba v configu je drahý model nebo vysoký effort jako výchozí.

Osobní, nebo týmový?

Rozhoduje, jestli to má platit pro každého v repu

Osobní ~/.codex

- Model a effort podle vkusu
- Profily: levny, hluboky, review
- Sandbox na Windows: podle verze notebooku
- Důvěra projektů (trust_level)
- MCP s osobním tokenem v proměnné

Týmový <repo>/.codex

- sandbox_mode a síť
- Filtr proměnných prostředí
- project_root_markers (repo bez Gitu)
- Rules: co smí bez ptaní (dotnet build, test)
- Mění se přes PR, revize jako kód

Týmový config jde vynutit jen přes requirements.toml. Bez něj je to dohoda, ne zákon.

Doporučený osobní config a profily

Výchozí levně, draze jen vědomě. Profil se zapíná při spuštění: codex -p <jméno>.

~/.codex/config.toml (výchozí)

```
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"
plan_mode_reasoning_effort = "high"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

[agents]
default_subagent_model = "gpt-6-luna"
default_subagent_reasoning_effort = "low"
max_concurrent_threads_per_session = 3

[shell_environment_policy]
inherit = "all"   # "core" rozbije dotnet
ignore_default_excludes = false
exclude = ["*CONNECTION*", "*PASSWORD*",
  "*SECRET*", "*TOKEN*", "*_PAT"]
```

Profily: ~/.codex/<jméno>.config.toml

```
# levny.config.toml   -> codex -p levny
model = "gpt-6-luna"
model_reasoning_effort = "low"

# hluboky.config.toml -> codex -p hluboky
model = "gpt-6.1-sol"
model_reasoning_effort = "high"
plan_mode_reasoning_effort = "xhigh"

# review.config.toml  -> codex -p review
sandbox_mode = "read-only"  # jen čte

# kurz.config.toml: síť jen pro cvičné repo
[sandbox_workspace_write]
network_access = true
```

Drahý režim je profil, který si člověk zapne. Ne výchozí stav, který zapomene vypnout.

Jak IT vynutí pravidla

requirements.toml: mantinely, které uživatel nepřebije

01 **Sepsat**

IT určí povolené režimy a zakázané příkazy

02 **Doručit**

přes GPO či Intune do ProgramData, nebo z cloudu

03 **Složit**

Codex při startu složí vrstvy configu

04 **Porovnat**

každou hodnotu s povolenými

05 **Opravit**

zakázanou přepne na povolenou a ohlásí to

Ochrana drží, když vývojář nemá admina a přihlašuje se firemním účtem ChatGPT.

requirements.toml: co v něm může být

Pro IT a správce workspace. Seznam povolených hodnot, ne nastavení.

%ProgramData%\OpenAI\Codex\requirements.toml

```
allowed_approval_policies = ["on-request"]
allowed_sandbox_modes =
  ["read-only", "workspace-write"]
allowed_web_search_modes =
  ["disabled", "cached"]

[windows]
allowed_sandbox_implementations =
  ["mxc", "elevated"]

[rules]
prefix_rules = [
  { pattern = [{ token = "git" },
     { any_of = ["push", "commit"] }],
    decision = "prompt" } ]
```

Co to znamená

```
- danger-full-access nejde zapnout
- schvalování nejde vypnout
- web jen z cache nebo vůbec:
  menší riziko prompt injection
- slabý sandbox unelevated
  nejde zvolit
- git push a commit vždy
  s potvrzením člověka

NEJDE vynutit:
- konkrétní model (jen povolit
  ve workspace)

Stejné jde doručit z cloudu:
Admin Console > Agent Security
```

Co musí platit vždy, patří sem. AGENTS.md doporučuje, requirements.toml vynucuje.

Sandbox: ohrada kolem příkazů agenta

Agent spouští build, testy a skripty. Sandbox určuje, kam smí sáhnout.



Zapisovat

jen do složky projektu a dočasných složek. .git a .codex jsou chráněné.



Číst

smí skoro všude, i mimo projekt. Pozor na soubory s hesly.



Síť

vypnutá. Ani NuGet, ani web, ani databáze.



Mimo ohradu

agent se zeptá. Vy schválíte, nebo zamítnete.

Chrání před chybou i před zmanipulovaným agentem. Bez sandboxu smí agent vše co vy.

Dvě nastavení: co smí a kdy se ptá

sandbox_mode je velikost ohrady, approval_policy je, kdy agent zaklepe

Hodnota	Co znamená	Kdy použít
sandbox_mode = "read-only"	agent jen čte, nic nezapíše ani nespustí s efektem	průzkum, mapa kódu, revize
sandbox_mode = "workspace-write"	zapisuje jen do projektu, síť vypnutá	běžná práce (doporučeno)
sandbox_mode = "danger-full-access"	bez ohrady, smí všechno co vy	nikdy na notebooku s přístupem k produkci
approval_policy = "on-request"	mimo ohradu se zeptá vás	výchozí, doporučeno
approval_policy = "never"	nikdy se neptá, co nejde, selže	jen skripty a CI v izolaci
--approve-for-me	místo vás schvaluje druhý model	když vás schvalování zdržuje

Agent se ptá na každý build? Povolte konkrétní příkazy přes rules. Ohradu nevypínejte.

Jak sandbox nastavit

Natrvalo v configu, jednorázově přepínačem, povolené příkazy v rules

config.toml (osobní nebo týmový)

```
sandbox_mode = "workspace-write"
approval_policy = "on-request"

[sandbox_workspace_write]
network_access = false
# další složka k zápisu (sdílená knihovna):
# writable_roots = ["C:\\src\\shared-lib"]

[windows]
sandbox = "mxc"   # nebo "elevated"
```

Jednorázově a rules

```
# jen pro jedno spuštění
codex -s read-only
codex -a on-request
codex -c windows.sandbox="unelevated"

# v běžícím Codexu
/permissions

# .codex/rules/dotnet.rules: bez ptaní
prefix_rule(pattern = ["dotnet", "build"],
            decision = "allow")
prefix_rule(pattern = ["dotnet", "test"],
            decision = "allow")

# ověření ohrady bez agenta
codex sandbox -- dotnet build
```

Síť je ve workspace-write vypnutá. dotnet restore spusťte ručně před agentem.

Na Windows: čím se ohrada postaví

Stejný sandbox_mode, tři technologie. Volí se klíčem [windows] sandbox.

Varianta	Jak funguje	Potřebuje	Síla ohrady
"mxc"	nativní izolace přímo z Windows	Win 11 24H2/25H2, Codex 0.162+	silná, doporučená
"elevated"	zvláštní omezený uživatel Windows a pravidla firewallu	IT jednou spustí codex sandbox setup --elevated	silná, síť opravdu zavřená
"unelevated"	příkazy pod vaším účtem s osekanými právy	nic	slabší: čte vše, síť hůř
WSL2	Linux uvnitř Windows	nic	pro msbuild, .NET Framework a VS nepoužitelné

Starší notebook bez 24H2: elevated s jednorázovou pomocí IT. Unelevated jen nouzově.

Hooky: kontrola, kterou agent nepřeskočí

Skript, který Codex spustí sám při určité události. Model o něm nerozhoduje.



Událost

Agent chce zapsat soubor, spustit příkaz nebo skončit.



Vstup

Codex pošle skriptu JSON: nástroj, co chce udělat, složku.



Rozhodnutí

Exit 0 akci pustí. Exit 2 ji zablokuje.



Důvod

Text ze stderr dostane agent a zkusí to jinak.

AGENTS.md je prosba, hook je kontrola. Co se nesmí stát nikdy, nepatří jen do textu.

Události: kdy hook běží a co umí

Šest nejužitečnějších z dvanácti

Událost	Kdy	Co hook umí	Příklad u vás
PreToolUse	před zápisem, příkazem, voláním MCP	zablokovat akci	zákaz úprav *.Designer.cs, golden dat, starých migrací
PostToolUse	po akci	ohlásit problém agentovi, akci nevrátí	změněné kódování cp1250, CRLF na LF
Stop	agent chce skončit	poslat ho zpátky k práci	bez výstupu testů úloha není hotová
UserPromptSubmit	po odeslání zadání	doplnit kontext, zadání zastavit	connection string v zadání
SessionStart	začátek vlákna	přidat kontext	větev, režim TESTY nebo IMPLEMENTACE
PermissionRequest	agent žádá o schválení	rozhodnout za člověka	zamítnout git push z agenta

Dalších šest: SessionEnd, PreCompact, PostCompact, SubagentStart, SubagentStop, Interrupt.

Kde hooky leží a kdo je upravuje

Soubor hooks.json, nebo sekce [hooks] v config.toml. Obsah je stejný.

Vrstva	Kde (Windows)	Kdo	Kdy běží
Osobní	%USERPROFILE%\.codex\hooks.json nebo config.toml	každý sám	vždy, po schválení v /hooks
Týmová	<repo>\.codex\hooks.json nebo .codex\config.toml	tým přes PR	jen v trusted projektu, po schválení
IT	spravovaná složka z requirements.toml ([hooks] windows_managed_dir)	IT	vždy, schválené předem
Plugin	hooks\hooks.json uvnitř pluginu	autor pluginu	když je plugin zapnutý

Vrstvy se nepřebíjejí, běží všechny. /hooks ukáže, co je načtené a co čeká na schválení.

Ukázka: hook proti úpravám generovaných souborů

Vlevo: kdy se hook spustí. Vpravo: co udělá.

<repo>\.codex\config.toml

```
[[hooks.PreToolUse]]
matcher = "^apply_patch$" # úprava souborů

[[hooks.PreToolUse.hooks]]
type = "command"
command = """powershell -NoProfile \
  -ExecutionPolicy Bypass \
  -File .codex/hooks/guard.ps1"""
timeout = 20          # sekund

# totéž jde zapsat jako JSON
# do .codex\hooks.json
```

<repo>\.codex\hooks\guard.ps1

```
# 1. Codex pošle, co chce agent udělat
$akce = [Console]::In.ReadToEnd() |
        ConvertFrom-Json
$text = $akce.tool_input | ConvertTo-Json

# 2. Mění soubor z designeru?
if ($text -match 'File: .*\.Designer\.cs') {
  # 3. Důvod pro agenta a stop
  [Console]::Error.WriteLine(
    "Soubor z designeru neměň. " +
    "Popiš změnu UI člověku.")
  exit 2
}
exit 0   # všechno ostatní projde
```

Důvod pište pro agenta, ne pro sebe. Dostane ho jako odpověď a podle něj pokračuje.

Co se stane, když hook zablokuje

Příklad: agent chce upravit Form1.Designer.cs

01

Agent chce zapsat

patch do souboru z designeru

02

Codex zavolá hook

JSON na stdin, agent zatím čeká

03

Hook rozhodne

cesta sedí na zákaz: exit 2

04

Důvod agentovi

text ze stderr místo výsledku zápisu

05

Agent pokračuje

jinou cestou, nebo se zeptá vás

Soubor zůstane netknutý. Agent ví proč a nemusí hádat.

Hooky: na co si dát pozor

Hook je brzda, ne brána. Bránou je pipeline.

1	**Schválení**	neschválený hook se přeskočí s varováním; nové schválení chce změna definice, změna skriptu nejspíš ne	*skripty hooků s povinným review, povinné hooky přes IT*
2	**Obcházení**	zápis shellem ([IO.File]::WriteAllText, copy) regulární výraz nepozná	*stejné pravidlo v pipeline a branch policy*
3	**Rychlost**	matcher ".*" spustí hook u každého kroku; výchozí timeout je 600 s	*timeout 20 s, matcher jen na potřebný nástroj*
4	**Chyba ve skriptu**	neparsovatelný vstup nebo výjimka: rozhodněte, jestli akci pustit, nebo zastavit	*otestovat bez modelu: echo JSON | powershell -File*
5	**Neověřené**	exit 2 v PostToolUse a Stop, spuštění změněného skriptu bez schválení, běh na Windows	*vyzkoušet na jednom stroji*

Každý hook má dvojče v pipeline. Hook šetří čas, pipeline chrání repo.

Skill: návod, který se načte, až když je potřeba

Složka se souborem SKILL.md. Postup pro určitý typ úlohy.

01

Start vlákna

agent dostane jen jména a popisy skillů

02

Úloha

zadání sedí na popis, nebo napíšete $jmeno

03

Načtení

teprve teď přečte celý SKILL.md

04

Podle potřeby

skripty a podklady ze složky skillu

05

Výsledek

stejný postup u každého v týmu

AGENTS.md stojí kontext v každém vlákně. Skill jen ve vlákně, kde se použije.

Anatomie skillu

Povinný je jen SKILL.md s name a description

.agents\skills\build-testy\SKILL.md

```
---
name: build-testy
description: Build a testy jednoho
  modulu. Použij po každé změně kódu
  a když máš ověřit, že testy prošly.
  Nepoužívej pro UI testy.
---
# Postup
1. Build: scripts\build.ps1 -Modul <M>
2. Testy: scripts\test.ps1 -Modul <M>
3. Výsledek čti z out\test-<M>.xml,
   ne z konzole.
4. Počet testů klesl? Zastav se.
5. Do odpovědi dej příkaz a souhrn.
```

Složka skillu a volby

```
.agents\skills\build-testy\
  SKILL.md              povinný
  scripts\build.ps1     agent spustí
  scripts\test.ps1
  references\           čte na vyžádání
  agents\openai.yaml    volby Codexu

# agents\openai.yaml
policy:
  allow_implicit_invocation: false
# = jen na výslovné $build-testy

# frontmatter smí mít jen: name,
# description, license,
# allowed-tools, metadata
```

Postup patří do kroků, ne do odstavců. Poslední krok říká, co agent musí odevzdat.

Kde skilly leží a jak se volají

Skilly se nepřebíjejí jako config. Agent vidí všechny najednou.

Vrstva	Kde (Windows)	Kdo	Na co
Repo	<repo>\.agents\skills, od složky, kde Codex běží, až po kořen	tým přes PR	build, testy, nová obrazovka, migrace
Osobní	%USERPROFILE%\.agents\skills (starší .codex\skills)	každý	vlastní zvyky a nástroje
IT	spravovaná složka správce	IT	firemní postupy pro všechny týmy
Vestavěné	součást Codexu	OpenAI	skill-creator a další

Volání: $jmeno, výběr v /skills, nebo samo podle popisu. Vypnutí: [[skills.config]] enabled = false.

Popis rozhoduje, jestli se skill použije

Agent vidí jen popis. Podle něj pozná, že skill patří k úloze.

Popis, který agent přehlédne

- „Testy“: kdy, jaké, pro co?
- „Užitečné nástroje pro ERP“
- Co skill obsahuje, ne kdy ho použít
- Dva skilly s podobným popisem

Popis, který funguje

- Kdy: „Použij po každé změně kódu v src/“
- Co: „Spustí testy modulu a přečte výsledek“
- Kdy ne: „Nepoužívej pro UI testy“
- Slova, která tým říká: mzdy, DPH, obrazovka

Skill, který se nepoužil, většinou nemá špatný postup. Má špatný popis.

Kam s pravidlem: od prosby po zákaz

Pět nástrojů. Čím níž v tabulce, tím méně záleží na tom, jestli model poslechne.

Nástroj	Co to je	Kdy platí	Příklad
AGENTS.md	text pro model	v každém vlákně	konvence, kde co je, jak se buildí
Skill	postup pro typ úlohy	když se úloha hodí nebo $jmeno	nová obrazovka, testy modulu
Hook	skript na událost	vždy, model ho neobejde	zákaz úprav *.Designer.cs a golden dat
Sandbox	ohrada procesů	vždy	zápis jen do projektu, bez sítě
requirements.toml	mantinely od IT	vždy, uživatel nepřebije	schvalování nejde vypnout

Čím víc záleží na tom, aby se něco nestalo, tím níž v tabulce to patří.

Co z notebooku odchází k OpenAI

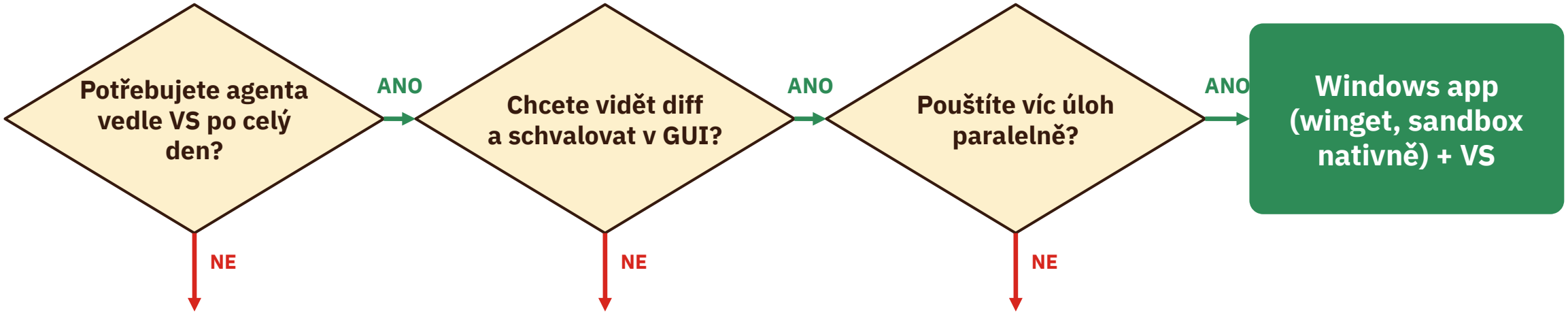
U mezd a účetnictví první otázka. Odpověď má být technická, ne „věřte nám“.

1	**Vše, co agent přečte**	obsah souborů, výstupy příkazů, logy, chybové hlášky; i web.config nebo appsettings s connection stringem	*nic v pracovní složce, co nesmí ven*
2	**Proměnné prostředí**	ve výchozím stavu agent vidí i proměnné s KEY, SECRET, TOKEN	*ignore_default_excludes = false, filtry*
3	**Data z databáze**	výsledky dotazů, které agent spustí nebo dostane přes MCP	*žádná dev DB s kopií zákaznických dat*
4	**Retence a trénink**	podle smlouvy workspace (Business, Enterprise); business data se ve výchozím stavu netrénují	*ověřit v Admin Console, ZDR*
5	**Lokální historie**	vlákna v %USERPROFILE%\.codex\sessions na disku	*[history] persistence, --ephemeral*

Hranici neurčuje instrukce, ale co je v pracovní složce, v env a v síti.

Visual Studio tým: kde má Codex běžet?

Oficiální rozšíření pro plné Visual Studio neexistuje



VS na kód a revizi, Codex vedle. Komunitní rozšíření do VS jen na vlastní riziko.

Váš setup: čtyři otázky

1. Jakou verzi Codexu a jaký výchozí model a effort máte? Liší se v týmu?
2. Kdy vás sandbox nejvíc zdržuje a proč?
3. Kolik trvá build a testy z příkazové řádky? Jde to vůbec?
4. Na co se agent ptá nejčastěji a co by mohl smět sám? Co musí rozhodnout IT?



BLOK 2 Z 7

Kontext pro miliony řádků

02

Agent nepotřebuje znát celé řešení. Potřebuje vědět, kde nehledat.

50 MINUT

Postup dne

Kolik z vašeho řešení agent uvidí

Hrubý odhad pro C#, ověřte si na svém repu příkazem z dalšího slidu

~10

tokenů na řádek C# (odsazení, identifikátory)

10–30 mil.

tokenů má řešení s 1–3 miliony řádků

< 5 %

se vejde do okna výchozího modelu (500k), i kdyby agent nic jiného nedělal

Zdroj: Odhad lektora; tokenizace se liší podle modelu

Nejde o to vejít se. Jde o to, aby agent četl ten správný zlomek.

Profil repozitáře za dvě minuty

PowerShell, spustit v kořeni. Výsledek patří do diskuze, ne do AGENTS.md.

Velikost a struktura

```
# ripgrep: winget install BurntSushi.ripgrep.MSVC
# řádky C# (rg respektuje .gitignore)
(rg --files -t cs | % { gc $_ } |
  Measure-Object -Line).Lines

# projekty (SDK-style i staré csproj)
rg --files -g "*.csproj" | measure

# největší soubory (kandidáti na výřez)
rg --files -t cs | % { [pscustomobject]@{
  f=$_; n=(gc $_).Count } } |
  sort n -desc | select -first 15
```

Co agent bude zbytečně číst

```
# generované soubory
rg --files -g "*.Designer.cs" | measure
rg --files -g "*.resx" | measure

# konce řádků (Git)
git ls-files --eol | Select-String "crlf" |
  measure

# soubory s BOM (zbytek: UTF-8, nebo 1250?)
rg -l --encoding none "(?-u)^\xEF\xBB\xBF" -t cs | measure

# čas buildu = čas zpětné vazby agenta
Measure-Command { dotnet build -bl }
# .NET Framework web: msbuild /m /bl /nr:false
```

Build přes 5 minut nebo testy přes 15 minut: agent bude ověřovat jen kousek. Řešte to první.

Strom instrukcí a nastavení

Co Codex načte z vaší složky a z repa, a jak to poskládá

OSOBNÍ · na vašem počítači

- `%USERPROFILE%\.codex\`
  - `config.toml` 5 — osobní nastavení
  - `levny.config.toml` 4 — profil, jen s codex -p levny
  - `AGENTS.md` 1 — vaše zvyky, jazyk
  - `rules\` `hooks.json` — schválené příkazy, hooky
- `%USERPROFILE%\.agents\skills\` — osobní skilly

REPO · společné pro tým

- `money-erp\` — kořen: složka s .git
  - `AGENTS.md` 2 — build, testy, zákazy, mapa
  - `.codex\` — jen v důvěryhodném projektu
    - `config.toml` 3 — týmové nastavení
    - `rules\` `hooks.json` — pravidla a hooky týmu
  - `.agents\skills\spust-testy\` — SKILL.md
  - `src\Ucetnictvi\` `AGENTS.md` 3
    - `Dph\` `AGENTS.md` 4 ← Codex spuštěný tady
  - `src\Mzdy\` `AGENTS.md` — *mimo cestu, nenačte se*

AGENTS.MD: SČÍTAJÍ SE

1 osobní → 2 kořen → 3 Ucetnictvi → 4 Dph

Spojí se v tomto pořadí, hlubší vyhrává. Projektové dohromady max 32 KiB.

CONFIG.TOML: VYŠŠÍ PŘEBIJE NIŽŠÍ

1. **IT: requirements.toml** jen omezí, uživatel nepřebije
2. **Přepínače: -m, -c** jen toto spuštění
3. **Repo: .codex\config.toml** týmové, v repu
4. **Profil: codex -p levny** jen když ho zapnete
5. **Osobní: config.toml** základ, platí vždy

Soubor obsahuje jen to, co mění. Zbytek zdědí, jinak výchozí hodnoty.

RULES A HOOKY

Platí všechny vrstvy najednou. U rules vyhrává nejpřísnější, hooky běží všechny.

SKILLY

Na startu jen jméno a popis. Celý SKILL.md až podle úlohy.

```
/debug-config    nastavení a odkud se vzalo
codex debug prompt-input    složené instrukce
```

Kořenový AGENTS.md: rozcestník, ne příručka

Do 120 řádků. Detail patří do modulů a skillů.

AGENTS.md (kořen repa, moduly jsou příklad)

```
# Money ERP: pravidla pro agenta
## Build a testy (vždy po změně)
- dotnet build Money.Core.slnf -c Debug
- <testovací nástroj> --filter <modul>
## Mapa (detail: docs/MAPA.md)
- Ucetnictvi/  doklady, DPH, uzávěrky
- Mzdy/        výpočty podle zákona
- Ui/          logika do presenterů,
                vzhled kontroluje člověk
## Nesahat
- *.Designer.cs, *.resx, Generated/
- db/migrations/ (jen nové soubory)
## Kódování
- zachovej kódování a konce řádků
## Hotovo znamená
- build bez nových warningů, testy
  zelené, diff jen v rozsahu úlohy
```

Proč takhle

```
- Načítá se v KAŽDÉM tahu:
  každý řádek platíte pořád
- project_doc_max_bytes 32 KiB:
  po dosažení se další AGENTS.md
  nepřidají (ověřte debug
  prompt-input)
- Solution filter (.slnf) =
  menší build, rychlejší smyčka
- Mapa jako odkaz: soubor se
  čte, až když je potřeba
- „Hotovo znamená“ je nejsilnější
  věta celého souboru
- Nic, co zjistí z kódu sám
  (jazyk, verze z csproj)
```

Každá věta v AGENTS.md má odpovídat na chybu, kterou agent už udělal.

AGENTS.md modulu: invarianty a pasti

Tady je doménová znalost, kterou model z internetu mít nemůže

src/Erp.Payroll/AGENTS.md (ukázka)

```
# Mzdy
## Invarianty
- Částky jen decimal. Zaokrouhlení jen
  přes CzkRounding, nikdy Math.Round.
- Parametry roku jen z IParametryMezd,
  nikdy konstanta v kódu.
- Výpočet nesmí číst DateTime.Now:
  období je vstup.
## Pravidlo na dvou místech
- Sazba je i v proceduře usp_...
  Změna = C# + SQL + test obou.
- T-SQL ROUND = od nuly,
  Math.Round = ToEven: rozejdou se.
## Testy
- Nová hodnota v testu = zdroj v
  komentáři (zákon, §, rok).
- golden/*.csv mění jen člověk.
```

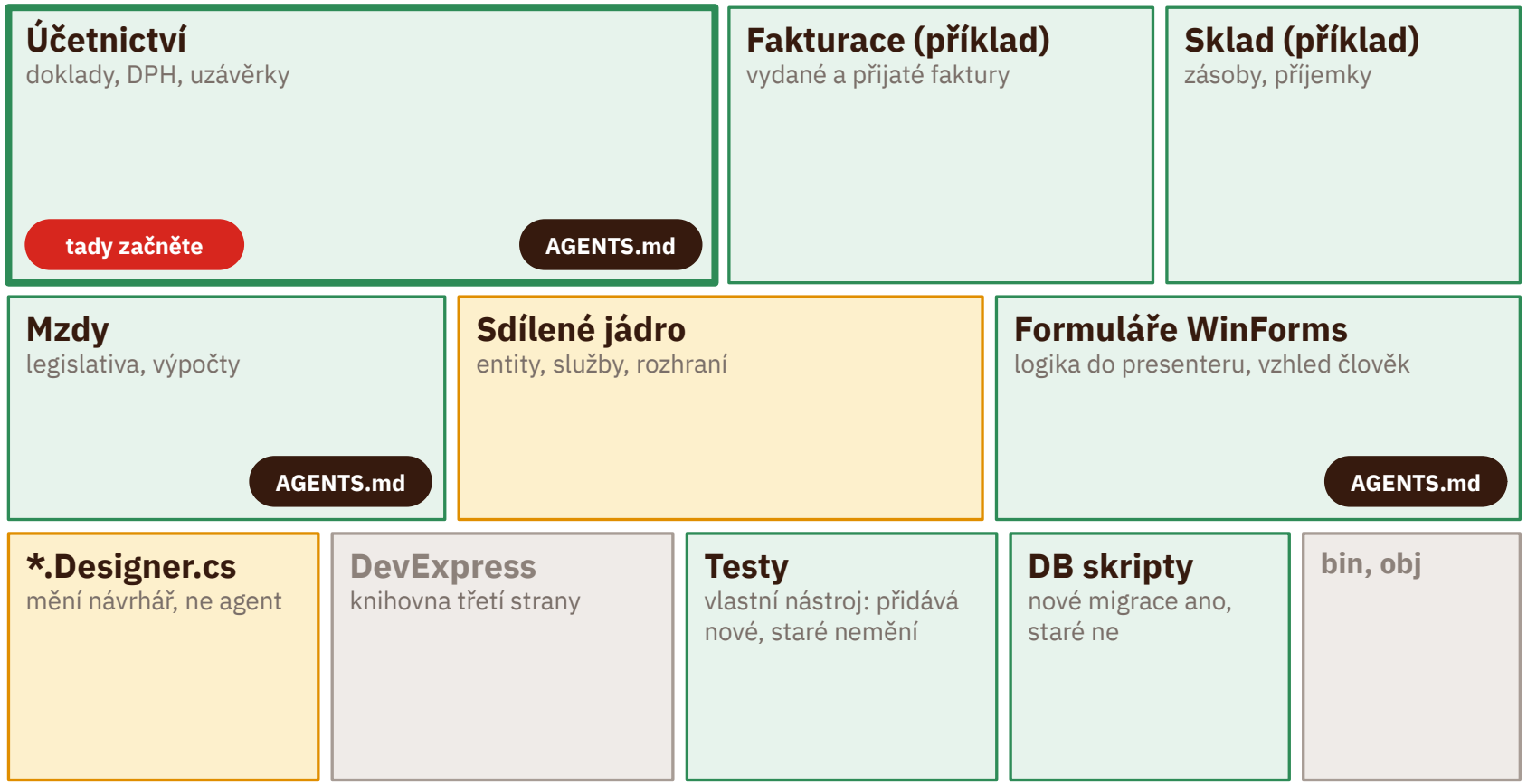
Co tam nepatří

```
- stav rozdělané práce
  (PROGRESS.md, BATCH.md)
- popis, co je v souborech vidět
- dlouhé výklady zákona:
  odkaz do docs/, ne kopie
- hesla, connection stringy,
  jména zákazníků
- pravidla jiného modulu
  (hlubší AGENTS.md vyhrává,
  ale nesčítá se s vedlejším)
```

Modul s vlastními pravidly má vlastní AGENTS.md. Modul bez nich ho nepotřebuje.

Mapa řešení pro agenta

Kde agent pracuje, co jen čte jako vzor a co nevidí



LEGENDA

- agent pracuje
- jen čte (vzor, schéma)
- mimo kontext

Pravidla výřezu

- V zadání: pracuj jen ve složce …
- Modul s vlastními pravidly má vlastní AGENTS.md
- Vzor ano: jeden typický soubor, ne celý modul
- Generovaný kód a knihovny do zákazů
- Mapu navrhne agent

Moduly jsou příklad (Fakturace a Sklad jsou vymyšlené), velikosti orientační. Skutečnou mapu postavíme spolu nad vaším repem.

Agent nepotřebuje znát celé řešení. Potřebuje vědět, kde pracovat a kde nehledat.

Jak agent najde ten správný zlomek

Od nejlevnějšího po nejdražší. Většinu práce udělají první tři řádky.

Nástroj	Co dá agentovi	Jak zavést
rg (ripgrep)	fulltext za milisekundy, respektuje .gitignore	má ho vestavěný, nic
docs/MAPA.md	modul → účel → vstupní třída → testy	agent vygeneruje, člověk opraví
docs/GLOSAR.md	česká doména → identifikátor v kódu	z dotazníku týmu, 50 pojmů stačí
Solution filter .slnf	build jen relevantních projektů	VS: Save As Solution Filter, nebo ručně (JSON)
git log -L, blame	proč je kód takový (záměr, ticket)	v zadání: „zjisti historii metody“
Roslyn / LSP přes MCP	reference, definice, typy jako kompilátor	pokročilé, jen pro velké refaktory
sys.sql_expression_dependencies	kdo volá proceduru, tabulku, pohled	výsledek dotazu jako soubor v repu
.rgignore	agent nehledá v generovaném kódu	**/Generated/**, *.Designer.cs

Glosář je nejlevnější investice: „sleva na poplatníka“ v zadání, identifikátor v kódu.

Hotspoty: kam agenta poslat nejdřív

Dluh je drahý jen tam, kde se kód mění (A. Tornhill)

Tři příkazy (Git historie)

```
git log --numstat --date=short `
  --pretty=format:'--%h--%ad--%aN' `
  --no-renames --after=2024-01-01 `
  --output=gitlog.txt -- src/

java -jar code-maat.jar -l gitlog.txt `
  -c git2 -a revisions -o churn.csv
java -jar code-maat.jar -l gitlog.txt `
  -c git2 -a coupling -o coupling.csv

# hotspot = churn × velikost
```

Co s tím

```
častá změna + složitý
  -> charakterizace, Mikado,
     agent PRVNÍ
zřídka + složitý
  -> jen číst, nesahat
coupling.csv
  -> „tyhle soubory se mění spolu“
  = pravidlo na dvou místech,
    které v AGENTS.md chybí

Bez Gitu: historii převést
(git-tfs) nebo exportovat
```

Top 20 hotspotů je backlog pro agenta. Zbytek milionu řádků počká.

Mapu nechte vygenerovat, ale opravte ji vy

Agent jen čte, nic nemění. Výsledek je soubor v repu.

Zadání (jen pro čtení)

```
› Jen čti, nic neměň. Projdi řešení
  a vytvoř docs/MAPA.md:
  - tabulka: modul | účel | vstupní
    třídy | testy | závislosti
  - u každého řádku 1 důkaz (soubor)
  - sekce „Nejisté“: co jsi odhadl
  - max 120 řádků
  Nepopisuj, co je zřejmé z názvu.
```

docs/MAPA.md: výstup po opravě

```
| Modul   | Účel           | Vstup        |
|---------|----------------|--------------|
| Mzdy    | čistá mzda,    | MzdaKalkulace|
|         | odvody, záloha|              |
| Vat     | rekapitulace, | DphRekapitul.|
|         | KH oddíly      |              |
| Data    | ADO.NET + SP   | IDbGateway   |

## Nejisté
- usp_KH_Podklady: volá ji i sestava?
- StareKonstanty.cs: kdo ji ještě čte?
```

Sekce „Nejisté“ je to nejcennější: ukazuje, kde agent hádá.

Kódování a konce řádků: tichý zabiják diffů

Typické pro české legacy repozitáře

Co se stane

- Soubor ve Windows-1250, agent uloží UTF-8
- Diff přes celý soubor, review nemožné
- Diakritika v řetězcích rozbitá až za běhu
- resx a SQL skripty se rozbijí taky

Co nastavit

- .editorconfig: charset po složkách (cp1250 neumí)
- .gitattributes: * text=auto, výjimky
- AGENTS.md: „zachovej kódování a EOL“
- Kontrola v PR: skript porovná kódování a EOL proti HEAD

Převod na UTF-8 po modulech, každý zvlášť; výjimky pro soubory, které čte jiný nástroj.

CVIČENÍ

Cvičení A: mapa a hierarchie instrukcí

erp-legacy, 20 minut

1. Nechte agenta (jen pro čtení) vygenerovat docs/MAPA.md podle zadání ze slidu, opravte sekci Nejisté

2. Napište kořenový AGENTS.md (do 120 řádků) a src/Erp.Payroll/AGENTS.md s invarianty

3. Nové vlákno: „Vysvětli, jak se počítá zdravotní pojištění, a kde jsou pravidla“. Šel rovnou na správné místo?



Na vlastním repu: totéž pro jeden modul, který znáte nejlíp. Necommitovat do hlavní linie.

BLOK 3 Z 7

Legacy vzory s agentem

03

Feathers, Fowler, Hammant: co z toho agent zvládne sám a kde ho musíte držet

100 MINUT

Postup dne

> **Legacy kód je prostě kód bez testů.**

Michael Feathers, Working Effectively with Legacy Code (2004)

Proč legacy kód agenta oslepí

Agent pracuje ve smyčce změna, build, testy, hotovo. Bez testů ta smyčka lže.

Kód s testy

- Test spadne: agent ví, že něco rozbil
- Test projde: změna nejspíš drží
- Agent opravuje, dokud není zeleno, a má pravdu
- Vy čtete diff a výsledky testů

Kód bez testů

- Build projde, testy nic nepokrývají: agent hlásí hotovo
- Divné chování „opraví“, i když je to požadavek zákazníka
- Význam kódu hádá z názvů („D“ = důchodce?)
- Vy čtete každý řádek, nebo věříte

Všechny vzory v tomhle bloku dělají jedno: dají agentovi zpětnou vazbu dřív, než změní kód.

Co agent v legacy kódu umí a co ne

Zobecnění ze všech vzorů tohoto bloku. Platí i pro práci mimo vzory.

Umí výborně

Najít a vyjmenovat
volání, místa parametru, závislosti: rychleji a úplněji než člověk, pokud je to v textu repa

Opakovat vzor
jeden ručně převedený kus zopakuje stokrát a konzistentně

Psát testy a kostry
charakterizační testy, loadery dat, porovnání, reporty rozdílů

Mechanické úpravy
rozhraní, adaptér, závislost do parametru, převod volajících; kompilátor hlídá

Vysvětlit a reprodukovat
co kód dělá, proč se dva výpočty liší a na kterém řádku

Neumí, nebo jen s dohledem

Rozhodnout, co je správně
chyba, nebo požadavek zákazníka? jaká tolerance? to je doména a zákon

Vidět, co není v repu
volající u zákazníků, VBA v binárním souboru, dynamické SQL, znalosti v hlavách

Zastavit se a vrátit
po neúspěchu opravuje dál a staví na rozbitém stavu

Udržet velkou změnu
čím větší diff, tím víc změn „mimochodem“ a tím méně jich člověk ověří

Poznat, kdy je hotovo
zelená v netestovaném kódu je pro něj stejná jako v testovaném

Pravidlo celého bloku: mechaniku agentovi, rozhodnutí člověku a mezi nimi strojová kontrola.

Proč tyhle vzory s agentem fungují

Šest vlastností, které mají společné. Podle nich poznáte vhodný přístup i mimo tento seznam.

Signál před změnou

Testy nebo porovnání vzniknou dřív, než se kód změní.

Malý krok, strojová kontrola

Každý krok končí příkazem: build, testy, diff, který člověk přečte.

Staré běží dál

Přepínač, fasáda nebo stará procedura. Chyba se vrací přepnutím.

Vzor místo popisu

Jeden ruční kus je zadání. Agent opakuje, nevymýšlí.

Rozhodnutí odděleně

Mechanika zvlášť, verdikt zvlášť. Člověk podepisuje jednu věc.

Stav v souboru

BATCH.md, MIKADO.md, ZJISTENI.md: práce přežije nové vlákno.

Když přístup tyhle vlastnosti nemá, agent v něm jen rychleji udělá chybu.

Co s agentem nefunguje: přístupy, kterým se vyhnout

S pečlivým člověkem občas projdou. S agentem selžou, protože je rychlý a neví, kdy přestat.

Přístup	Proč s agentem selže	Místo toho
Velký přepis naráz („převeď modul na .NET 10“)	žádný mezistav k ověření, tisíce řádků diffu, review jen naoko	Strangler Fig, Branch by Abstraction po krocích
Refaktoring bez testů („uklid to“)	zelená nic neznamená, změnu chování nepoznáte	nejdřív charakterizační testy
Oprava a refaktoring v jednom kroku	v diffu nejde odlišit chtěnou změnu od nechtěné	dvě zadání, dva commity
Testy i kód píše agent sám	testuje vlastní pochopení, chyba je v obou stejně	hodnoty od člověka, golden data, paralelní běh
Dlouhá feature větev	agent píše rychle, rozdíl roste, nic neběží v provozu	Branch by Abstraction v hlavní větvi
„Agent rozhodne sám“	toleranci, význam kódu nebo snapshot nastaví tak, aby bylo zeleno	rozhodnutí v zadání, ADR nebo u člověka
Celá úloha v jednom vlákně	kontext se plní, každý tah je dražší, začátek zapomíná	stav v souboru, krátká vlákna
Agent proti sdílené nebo ostré DB	chyba je nevratná a dopadne na ostatní	skript v db/, spouští člověk

Společný jmenovatel: velký krok bez kontroly. Agent ho udělá rychleji, ale ne lépe.

Mapa vzorů: co svěřit agentovi a co ne

Ke každému řádku je dál karta. Poslední sloupec: jak velkou část vzoru zvládne agent.

Vzor	Co to je, jednou větou	Agent udělá	Drží člověk	Pro agenta
Sprout a Wrap	nový kód vedle staré metody, ne doprostřed	metodu s testy, jedno volání	starou metodu netknutou	výborně
Seam	místo, kde jde podstrčit jinou závislost (čas, DB)	parametr, rozhraní, přetížení	stejné chování	dobře
Charakterizační test	test, který zapíše dnešní chování, i chybné	stovky případů z dat	co je chyba	výborně
Golden master	celý výstup jako schválený soubor	snapshot, odstranění datumů	schválení snapshotu	dobře
Dávky	stejná změna na stovkách míst, po kouscích	dávku podle vzoru	vzor a kontrolu dávky	výborně
Branch by Abstraction	rozhraní před starým, nové vedle, přepínač	rozhraní, adaptér, nový kód	přepnutí, smazání	po krocích
Paralelní běh	starý a nový výpočet na stejných datech	kostru a report rozdílů	každý rozdíl	kostra ano
Strangler Fig	nová část přebírá funkce staré po jedné	fasádu, převod jedné části	pořadí, vypnutí staré	jen jeden krok
Anti-Corruption Layer	překlad mezi starou DB a novým modelem	mapování z číselníku	význam kódů	dobře
Expand–Contract	změna DB ve fázích: přidat, převést, ubrat	skripty po fázích	kdy smí contract	expand, migrate
Mikado	velká změna jako strom malých kroků	pokusy, zápis, návrat	cíl, kdy přestat	se zadáním

Agent dělá mechaniku. Co je ve sloupci „Drží člověk“, rozhoduje člověk.

Feathersův postup změny v legacy kódu

Pět kroků z roku 2004. Agent je dělá rychle, ale jen když je v zadání máte vy. Detail na dalším slidu.

01

Místa změny

agent najde, kde se změna projeví (rg, volání, SQL)

02

Místa testu

kde jde chování pozorovat bez UI a DB

03

Rozbít závislosti

seam: rozhraní, parametr, virtuální metoda

04

Testy

charakterizační: zachytit DNEŠNÍ chování, nic neměnit

05

Změna a refaktoring

malé kroky, testy po každém

Agent nejčastěji přeskočí kroky 2 až 4. Zadání je musí vynutit.

Feathers krok za krokem: co agent udělá a co přeskočí

Každý krok je samostatné zadání. Věta v posledním sloupci ho vynutí.

Krok	Co to znamená	Agent sám od sebe	Věta do zadání
1 Místa změny	najít všechna místa, kde se změna projeví: C#, SQL, VBA, konfigurace	rg najde volání za minutu; dynamické SQL, reflexi a volání z Accessu nevidí	„Vypiš všechna místa, kde se X počítá nebo čte, i v db/. Nic neměň.“
2 Místa testu	bod, kde jde výsledek zachytit bez UI a DB (pinch point: jedno místo pokryje víc změn)	testuje přes UI nebo DB, nebo každou drobnou metodu zvlášť	„Navrhni, kde výsledek zachytit bez UI a DB, a počkej na mě.“
3 Rozbít závislosti	udělat kód zavolatelným z testu: čas, statika a DB jako parametr	mechaniku zvládne, „mimochodem“ opraví i logiku	„Jen nejmenší změna pro test. Starý podpis zachovej, chování neměň.“
4 Charakterizační testy	testy, které zapíšou dnešní chování včetně chyb	píše rychle, ale upraví kód nebo hodnotu, aby test prošel	„Očekávané hodnoty z AKTUÁLNÍHO kódu. Podezřelé do ZJISTENI.md.“
5 Změna	teprve teď, malými kroky, testy po každém	velký diff najednou, změní test místo kódu	„Testy z kroku 4 neměň. Po každém kroku build a testy.“

Bez těch vět agent skočí z kroku 1 rovnou na 5 a nahlásí hotovo.

Feathers na erp-legacy: výpočet mezd

Pět kroků na kódu z cvičení B a C

Krok	Co v repu najdete	Kde agent uklouzne
1 Místa změny	MzdaKalkulace.Vypocitej; volají ji MzdyZpracovani, MzdaVypocetForm a HlavniForm (legacy-winforms/); zdravotní znovu počítá procedura usp_Mzdy_Vypocet_Uloz	hledá jen v src/: proceduru v db/ a HlavniForm nenajde
2 Místa testu	Vypocitej(zam, hruba, obdobi) vrací celý MzdovyListek bez UI a DB	testuje přes formulář nebo jen čistou mzdu
3 Závislosti	konstruktor bez parametrů bere ServiceRegistry, druhý přijme IParametryMezd: seam už existuje	přepisuje ServiceRegistry v celém řešení
4 Charakterizace	golden/mzdy-2026.csv, 52 řádků, 11 z nich podezřelých	„opraví“ hranici 23 % v kódu; 31250,5 čte s tečkou
5 Změna	cvičení C: INetWageCalculator, nový výpočet vedle starého	přepíše MzdaKalkulace místo obalení

Kroky 1 až 3 jsou čtení, ne psaní. Nechte agenta nejdřív jen hledat a hlásit.

Sprout a Wrap: nový kód vedle starého

Feathers, kap. 6: když musíte přidat funkci a nestihnete otestovat celou metodu

CO TO JE

Novou funkci nepíšete doprostřed obří metody. Sprout: nová metoda s vlastními testy a ve staré metodě jen jeden řádek, který ji zavolá. Wrap: starou metodu přejmenujete a nová se stejným jménem ji zavolá a přidá chování před ní nebo po ní.

1 **Najít místo ve staré metodě** → 2 **Nová metoda + testy** → 3 **Jeden řádek volání** → 4 **Zbytek beze změny**

AGENT ZVLÁDNE

- Napsat novou metodu i s testy, bez UI a DB
- Vložit jedno volání na správné místo
- Wrap: přejmenovat a přesměrovat i stovky volajících, kompilátor hlídá

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Novou logiku vepíše doprostřed obří metody, je to nejkratší cesta
- „Když už tam je“, uklidí okolní kód: diff přes celou metodu
- Nová metoda sáhne na stejnou statiku (ServiceRegistry, DateTime.Now) a test nejde napsat

DO ZADÁNÍ NEBO AGENTS.MD

```
Novou funkci napiš jako
samostatnou metodu
s vlastními testy.
Do Vypocitej přidej jen
jeden řádek, který ji volá.
Zbytek Vypocitej neměň.

Hotovo: git diff ve
Vypocitej = 1 řádek.
```

Sprout je nejlevnější cesta, jak do netestovaného kódu dostat testovaný. Agent ji sám nezvolí.

Seam: místo, kde jde změnit chování bez editace kódu

Krok 3 Featherse v kódu: závislost ven do parametru, chování beze změny

Před: netestovatelné

```
public decimal SpocitejZalohu(int zamId)
{
  var z = ServiceRegistry
    .Get<IZamestnanci>().Nacti(zamId);
  var rok = DateTime.Now.Year;
  var sazby = Parametry.Pro(rok);
  ...
  Trace.Log("zaloha " + z.Id);
  return Math.Round(dan, 0);
}
```

Po: seam přes parametr a rozhraní

```
// .NET 8+: TimeProvider; .NET Framework
// 4.6.2+: balík Microsoft.Bcl.TimeProvider
public decimal SpocitejZalohu(
  Zamestnanec z,
  IParametryMezd sazby)
{
  ...
  return Math.Round(dan, 0);
  // BEZE ZMĚNY: oprava zaokrouhlení
  // je samostatná položka s testem
}
// starý vstupní bod zůstává a deleguje
```

Zadání: „vytáhni závislosti do parametrů, starý podpis zachovej a deleguj, chování neměň“.

Math.Round: past, kterou agent zopakuje

Výchozí zaokrouhlení v .NET je bankéřské, zákon počítá jinak

Co agent napíše „správně“

```
› csi   (C# Interactive, Developer PowerShell)

Math.Round(2.5m)           // 2  (ToEven)
Math.Round(3.5m)           // 4
Math.Round(2.5m, MidpointRounding
  .AwayFromZero)           // 3

// pojistné: zaokrouhlit NAHORU
Math.Ceiling(23_456.10m * 0.135m)
// = 3 167, ne 3 166,57

// .NET Core 3.0+, NE v .NET Framework:
MidpointRounding.ToPositiveInfinity
// T-SQL ROUND(2.5, 0) = 3 (od nuly)
```

Co musí být v instrukcích modulu

```
# AGENTS.md (modul Mzdy)
## Zaokrouhlování
- Nikdy Math.Round bez MidpointRounding.
- Pojistné a záloha: nahoru na Kč
  (Math.Ceiling), viz docs/PARAMETRY.
- Haléřové částky jen decimal,
  nikdy double/float.
- T-SQL ROUND = od nuly, Math.Round
  = ToEven: C# a SQL se rozejdou.
- Změna zaokrouhlení = nový test
  s hodnotou od člověka.
```

Model zná C# z internetu. Váš zákon a vaše zaokrouhlování z internetu nezná.

Charakterizační test: zapsat, co kód dělá dnes

Feathers, kap. 13. Nejlepší první úloha pro agenta v legacy kódu.

CO TO JE

Test neověřuje, co je správně, ale co kód dělá teď. Spustíte starý kód na reálných vstupech a jeho výsledky zapíšete jako očekávané hodnoty. Když pak změna něco posune, test to řekne. Zapisují se i chybné výsledky: oprava chyby je samostatná úloha.

1 Vstupy z reálných dat → **2 Spustit starý kód** → **3 Výsledek = očekávaná hodnota** → **4 Podezřelé zapsat zvlášť**

AGENT ZVLÁDNE

- Stovky případů z CSV nebo z anonymizované kopie dat za minuty
- Hraniční případy: nula, hranice pásem, záporné částky
- Loader dat, [Theory] přes všechny řádky, report

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Opraví produkční kód podle zákona, aby test „dával smysl“
- Podezřelé řádky označí Skip nebo z testu vynechá
- Porovná jedno číslo (čistou mzdu), zbytek výsledku nehlídá

DO ZADÁNÍ NEBO AGENTS.MD

```
Očekávané hodnoty ber
z AKTUÁLNÍHO kódu.
Produkční kód NEMĚŇ.
Podezřelé výsledky piš
do ZJISTENI.md, ne do testu.
Porovnej celý výsledek,
ne jedno pole.

Hotovo: diff jen v tests/.
```

Jestli je výsledek chyba, rozhoduje člověk se zákonem v ruce. Agent ji najde a zapíše.

Zadání: charakterizační testy

Nejlepší první úloha pro agenta v legacy kódu

```
› Napiš charakterizační testy pro
  MzdaKalkulace.Vypocitej.
  - Vstupy ber z golden/mzdy-2026.csv,
    očekávané hodnoty z AKTUÁLNÍHO kódu.
  - Produkční kód NEMĚŇ (ani formát).
  - Podezřelé výsledky nepiš do testu
    jako správné: zapiš je do ZJISTENI.md
    s řádkem CSV a důvodem.
  - Spusť dotnet test, musí projít.
  - Hotovo: testy zelené, git diff
    jen v tests/ a ZJISTENI.md.
```

- **AKTUÁLNÍ kód**
  test popisuje realitu, ne zákon
- **NEMĚŇ**
  jinak test nic nechrání
- **ZJISTENI.md**
  chyby jako nové položky backlogu
- **git diff**
  hotovo se kontroluje strojově

Charakterizační test neříká, co je správně. Říká, co se změnilo.

Golden master: celý výstup jako schválený soubor

Pro sestavy, exporty a XML podání, kde by ruční asserty nikdy neskončily

CO TO JE

Místo desítek assertů uložíte celý výstup (výplatní páska, XML kontrolního hlášení) jako schválený soubor. Test příště vyrobí nový výstup a porovná ho se schváleným. Rozdíl ukáže jako diff. V .NET to dělá knihovna Verify: nový výstup je .received, schválený .verified.

1 Vyrobit výstup → 2 Odstranit proměnlivé (datum, GUID) → 3 Člověk schválí → 4 Příště porovnat

AGENT ZVLÁDNE

- Napsat test a scrubbery pro datumy, GUID a časy
- Vysvětlit rozdíl v diffu lidsky
- Nahradit stovky ručních assertů jedním snapshotem

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Test spadne a agent snapshot přeschválí: .received přejmenuje na .verified a hlásí zeleno
- Scrubber smaže i pole, které se měnit nesmí
- Snapshot vyrobí z výstupu, který nikdo nečetl

DO ZADÁNÍ NEBO AGENTS.MD

```
.verified.* NIKDY
nepřepisuj ani nepřejmenuj.
Když se výstup liší,
zastav se a ukaž diff.

# a v pipeline:
# změna *.verified.*
# = povinný reviewer
```

Schválení snapshotu je podpis člověka. Agent smí vyrobit jen .received.

Schvalovací testy (snapshot) s Verify

Velké výstupy jako schválený snapshot. Váš vlastní nástroj umí totéž, pravidla jsou stejná.

Test (xUnit + Verify.Xunit)

```
[UsesVerify]  // ve starších verzích
public class KhExportTests
{
  [Fact]
  public Task Export_KH_rijen()
  {
    var xml = KhExport.Vytvor(
      Data.Rijen2026());
    return VerifyXml(xml)
      .ScrubLinesContaining("DatumVytvoreni")
      .UseDirectory("Snapshots");
  }
}
```

Co se commituje a co ne

```
# vznikne při běhu
KhExportTests.Export_KH_rijen
  .received.xml   # nový výstup
  .verified.xml   # schválený

# .gitignore
*.received.*

# .gitattributes
*.verified.* text eol=lf

# pravidlo pro agenta:
# .verified.* NIKDY nepřepisuj
```

Agent smí vytvořit .received. Přejmenovat na .verified smí jen člověk.

Velká změna po dávkách

Jedno zadání, seznam v souboru, kontrola každé dávky

Legacy modul
stovky souborů, testy řídké

BATCH.md
seznam dávek, stav, jedno zadání pro všechny

Dávka 1	Dávka 2	Dávka 3	Dávka 4	Dávka 5
vzor ručně + 3 soubory	10 souborů	10 souborů	10 souborů	zbytek
build	build	build	build	build
testy	testy	testy	testy	testy
diff	diff	diff	diff	diff
hotovo, commit	hotovo, commit	čeká na diff	*čeká*	*čeká*

Záchranná síť: charakterizační testy a build s analyzátory před první dávkou

Zachytit testy současného chování → **Změnit** jedna dávka, stejné zadání → **Ověřit** build, testy, diff do 400 řádků → **Uložit** commit, stav do BATCH.md

Velikost dávky určuje, kolik diffu stihne člověk přečíst. Ne kolik zvládne agent.

Branch by Abstraction: výměna bez dlouhé větve

P. Hammant (2007), M. Fowler (2014). Na tom stojí cvičení C.

CO TO JE

Před starou implementaci postavíte rozhraní. Všichni volající jdou přes rozhraní, stará třída zůstává beze změny. Vedle vznikne nová implementace a přepínač vybere, která běží. Všechno je v hlavní větvi a každý krok jde vydat.

1 **Rozhraní nad starým** → 2 **Volající přes rozhraní** → 3 **Nová implementace** → 4 **Přepínač** → 5 **Smazat staré**

AGENT ZVLÁDNE

- Rozhraní a adaptér nad starou třídou
- Přesměrovat stovky volajících, kompilátor hlídá
- Novou implementaci s vlastními testy

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Dostane všech pět kroků naráz: 800 řádků diffu
- Přepíše starou třídu místo obalení
- Přepínač udělá jako #if nebo bool natvrdo a krok 5 nikdy nepřijde

DO ZADÁNÍ NEBO AGENTS.MD

```
Udělej jen krok 2:
volající přes
INetWageCalculator.
Chování stejné, testy
z cvičení B zelené.
MzdaKalkulace NEMĚŇ.
Pak se zastav.

# krok 5 = položka
# v backlogu s datem
```

Každý krok je samostatný pull request se zelenými testy. Agent dostane vždy jen jeden.

Datum účinnosti není přepínač

Přepínač řídí, který kód běží. Datum účinnosti řídí, které právo platí.

Verze pravidla s platností (C# 7.3)

```
interface IPravidloSlevy {
  DateTime PlatnostOd { get; }
  decimal Sleva(Poplatnik p, Obdobi o);
}
// výběr podle období, ne podle přepínače:
IPravidloSlevy Pro(Obdobi o) =>
  vse.Where(p => p.PlatnostOd <= o.Prvni)
     .OrderByDescending(p => p.PlatnostOd)
     .First();

// prosinec 2026 přepočítaný v roce 2028
// = pravidla 2026
```

Přepínače podle Hodgsona a audit

```
release       nedokončený kód, týdny
ops           kill switch: zpět na legacy
permissioning pilotní firmy, licence
experiment    A/B: u mezd ne

Auditní stopa každého výsledku:
- verze kódu, hash parametrů,
  stav přepínačů, vstupy, čas UTC
- parametry: temporal table
- výsledky: ledger (SQL 2022+)
```

Kdo zamění přepínač a datum účinnosti, přepočítá loňské mzdy podle letošního zákona.

Paralelní běh: starý a nový počítají totéž

Jediný důkaz, který přesvědčí metodika u mezd, DPH a uzávěrek

CO TO JE

Starý i nový výpočet běží na stejných datech. Výsledek se bere vždy ze starého, nový se jen porovnává. Každý rozdíl jde do reportu. Přepnout na nový smíte, až rozdíly zmizí, nebo je člověk vysvětlí jako chybu starého kódu.

1 **Stejný vstup** → 2 **Starý: platí** → 3 **Nový: jen porovnat** → 4 **Report rozdílů** → 5 **Člověk rozhodne**

AGENT ZVLÁDNE

- Kostru porovnání (Scientist.NET nebo vlastní)
- Report rozdílů po kategoriích
- Ke každému rozdílu najít řádek a reprodukci

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Nastaví toleranci ±1 Kč a rozdíly zmizí
- Vrací nový výsledek místo starého
- Rozdíl „vyřeší“ tak, že nový kód zkopíruje chybu starého

DO ZADÁNÍ NEBO AGENTS.MD

```
Vracej VŽDY starý výsledek.
Tolerance 0 Kč.
Rozdíl neopravuj: zapiš
řádek, obě hodnoty
a hypotézu proč.
Rozhoduje člověk.
```

Nový výpočet jde do provozu, až paralelní běh mlčí. Ne když agent řekne, že je hotovo.

Paralelní běh v kódu: Scientist.NET, nebo vlastní report

Pro mzdy, DPH a uzávěrky: jediný přesvědčivý důkaz pro skeptiky

Scientist.NET (port GitHub Scientist)

```
var vysledek = Scientist.Science<decimal>(
  "cista-mzda", e =>
{
  e.Use(() => legacy.Vypocti(vstup));
  e.Try(() => nova.Vypocti(vstup));
  e.Compare((a, b) => a == b);
  e.AddContext("zam", vstup.Id);
});
// vrací VŽDY výsledek Use (starý),
// rozdíly jdou do publisheru
```

Nebo vlastní rekonciliace nad golden daty

```
› dotnet test --filter `
   "FullyQualifiedName~StranglerTests.Paralelni_beh"
  # report = tests/.../Approved/*.approved.txt

  52 případů
  41 shoda
  11 rozdílů:
    8× záloha (vysoké mzdy)
    3× daňový bonus (nízké mzdy)

# každý rozdíl = rozhodnutí člověka:
# chyba nového, nebo chyba starého?
```

Rozdíl není chyba nového kódu. Je to otázka, kterou musí rozhodnout člověk se zákonem v ruce.

Paralelní běh v praxi: tolerance 0, osobní údaje

Důkaz, který podepíše metodik i pověřenec pro ochranu osobních údajů

Tok

```
uzavřená období
  -> anonymizace (hash se solí mimo repo;
     datum narození, děti, ZTP/P zůstávají)
  -> kontrola: legacy na originále
     = legacy na anonymu
  -> paralelní běh starý vs. nový
  -> tabulka rozdílů
  -> triáž (metodik)
  -> report za firmu a období
```

Tabulka rozdílů a pravidla

```
audit.ParallelRunDiff
  Firma, Obdobi, Pole, Legacy, Novy
  VerzeKodu, VerzePravidel
  Kategorie: ZAOKROUHLENI | CISELNIK
    | CHYBA_LEGACY | CHYBA_NOVY
  Vysvetleni, SchvalilMetodik

- tolerance 0 Kč u zákonných výstupů
- součty za firmu a období odhalí
  systematickou chybu
- agent: kostra, triáž, reprodukce;
  NIKDY neanonymizovaná data
```

Rozdíl je otázka pro metodika. Agent ji umí dobře položit, ne zodpovědět.

Strangler Fig: nové postupně přeroste staré

M. Fowler (2004). Podle fíkovníku, který obroste hostitelský strom.

CO TO JE

Starý systém nepřepisujete najednou. Před něj dáte fasádu (rozcestník), která posílá každý požadavek buď do staré, nebo do nové části. Nová část přebírá funkce jednu po druhé. Když stará už nic nedělá, vypne se.

1 Fasáda před starým → **2 Jedna funkce do nového** → 3 Paralelní běh → 4 Přepnout cestu → 5 Smazat staré

AGENT ZVLÁDNE

- Postavit fasádu a směrování
- Převést jednu funkci podle vzoru i s testy
- Najít, co ještě volá starou cestu

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Převede víc funkcí najednou, „je to podobné“
- Starou cestu nechá „pro jistotu“: dvojí údržba navždy
- Nezná váš framework: bez vzoru vymýšlí, jak se tu píše obrazovka

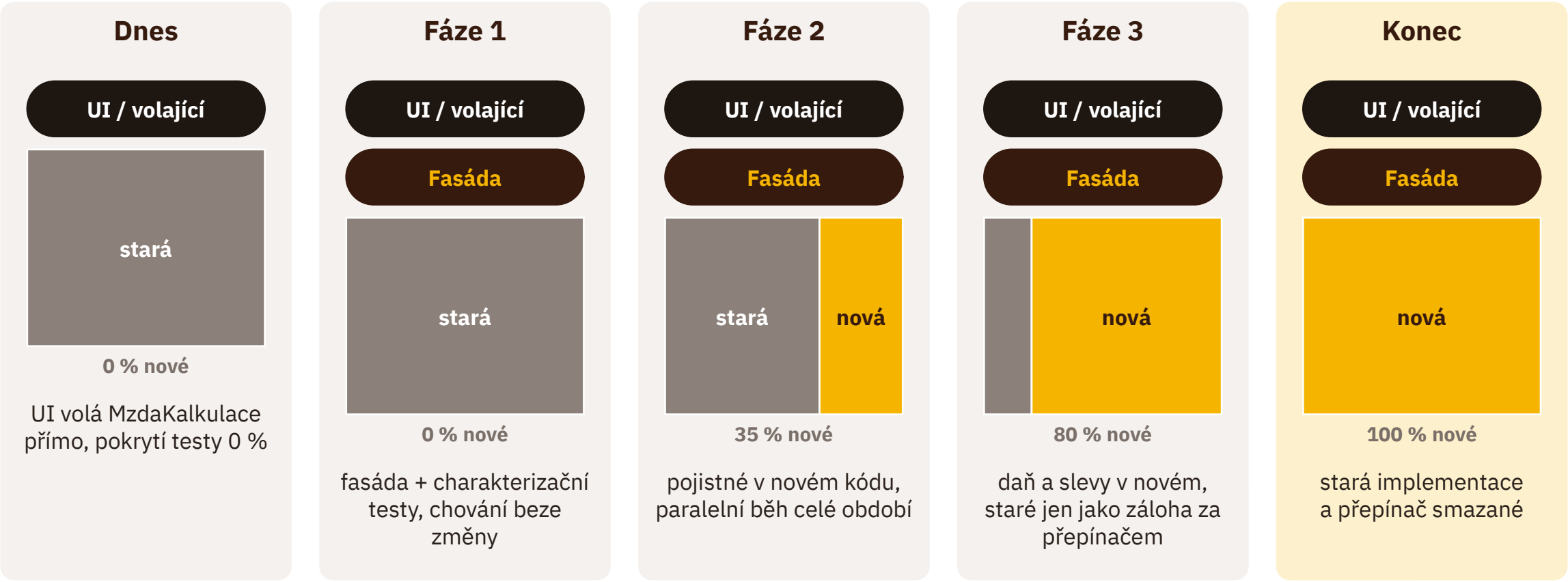
DO ZADÁNÍ NEBO AGENTS.MD

```
Převeď jen <funkci>
podle vzoru <soubor>.
Fasáda posílá na nové
jen <cestu>, zbytek
na staré beze změny.
Přidej test, který zavolá
starou i novou a porovná.
```

Pořadí převodu a datum vypnutí staré části určuje architekt. Agent dělá jeden krok.

Strangler Fig: modul po modulu

Ilustrace: fasáda směruje, nová implementace roste kolem starého jádra



Strangler bez data, kdy se stará část smaže, jen zdvojí údržbu.

Anti-Corruption Layer: překladač mezi starým a novým

E. Evans, Domain-Driven Design (2003). Chrání nový kód před zvyky staré databáze.

CO TO JE

Stará databáze má své zkratky: sloupec TYP_PP s hodnotou „D“, MZDA_ZAKL, NULL místo nuly. Nový kód s nimi nepracuje přímo. Mezi ně dáte jednu vrstvu, která staré řádky převádí na nové typy (Uvazek.Dpp) a zpět. Staré názvy zná jen tahle vrstva.

1 Starý řádek z DB → **2 Překlad v ACL** → **3 Nový model** → **4 Neznámý kód = výjimka**

AGENT ZVLÁDNE

- Mapování sloupců a typů, i stovky polí
- Testy mapování z číselníku v repu
- Test architektury: nic mimo ACL nevidí Legacy* typy

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Vymyslí význam kódu: „D asi důchodce“
- Neznámou hodnotu pošle do default větve místo chyby
- Pohodlně použije starý typ i v novém kódu

DO ZADÁNÍ NEBO AGENTS.MD

```
Význam kódů ber jen
z ciselniky/*.csv.
Neznámý kód = výjimka
UnknownLegacyCode,
nikdy default.
Legacy* typy jen v Acl/.
```

Agent je skvělý v mapování a špatný v hádání. Číselník mu dejte jako soubor.

Anti-Corruption Layer a bublina pro agenta

Nový model nad starou databází, aniž by ho stará databáze nakazila

ACL v kódu (C# 7.3)

```
static Zamestnanec ToDomain(LegacyZamRow r)
{
  Uvazek u;
  switch (r.TYP_PP) {
    case "H": u = Uvazek.Hpp; break;
    case "D": u = Uvazek.Dpp; break;
    case "C": u = Uvazek.Dpc; break;
    default: throw new UnknownLegacyCode(
      "TYP_PP", r.TYP_PP); // nikdy default
  }
  return new Zamestnanec(r.ID_ZAM, u,
    Penize.Czk(r.MZDA_ZAKL ?? 0m));
}
```

Bublina: nejbezpečnější místo pro agenta

```
src/Bubbles/CestovniNahrady/
  AGENTS.md  pracuješ jen tady,
             mimo bublinu jen čtení
  Acl/       jediné místo s Legacy* typy
  Model/     čistý model, vlastní testy
  ciselniky/ TYP_PP.csv (čte, nehádá)

CI (ne prompt):
- změna mimo bublinu = review
- NetArchTest: nic mimo *.Acl
  nereferencuje Legacy*
```

Bublina je malá, má pravidla a hlídá ji CI. Pro první agentní modul ideální.

Expand–Contract: změna databáze ve třech fázích

Také Parallel Change (D. Sato, 2014). Stará i nová forma žijí chvíli vedle sebe.

CO TO JE

Proceduru nebo sloupec neměníte najednou. Expand: přidáte novou formu vedle staré (nový parametr s výchozí hodnotou NULL, nebo procedura _v2). Migrate: volající postupně přejdou na novou. Contract: starou formu smažete, až ji nikdo nevolá. Každá fáze je samostatné nasazení.

1 Expand: přidat nové → **2 Migrate: převést volající** → **3 Contract: smazat staré**

AGENT ZVLÁDNE

- Skripty všech fází jako soubory v db/
- Najít volající v C#, SQL i VBA (rg a sys.sql_expression_dependencies)
- Test proti LocalDB, rekonciliační dotaz

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Expand i contract v jednom skriptu: starý import přestane fungovat
- Nový parametr dá doprostřed seznamu
- Při NULL „opraví“ i starý výpočet: to už není expand

DO ZADÁNÍ NEBO AGENTS.MD

```
Jen fáze EXPAND.
Nový parametr na konec,
výchozí NULL. Při NULL
původní chování.
Contract jen jako soubor
se seznamem podmínek.
Nic nespouštěj proti DB.
```

Contract je rozhodnutí s datem, ne další krok agenta.

Expand–Contract pro uloženou proceduru

Pravidlo je v C# i v SQL a liší se: typická legacy situace

Fáze (každá zvlášť nasaditelná)

```
-- 1 EXPAND: nová procedura vedle
CREATE PROCEDURE usp_KH_Podklady_v2 ...
-- (lehčí: nový parametr = NULL na konci)

-- 2 MIGRATE: C# volá v2 za přepínačem
--   paralelní běh v1 vs v2 na kopii DB

-- 3 CONTRACT: až nikdo nevolá v1
DROP PROCEDURE usp_KH_Podklady;
-- jméno _v2 zůstává; NIKDY sp_rename

-- kdo na proceduru ještě odkazuje:
SELECT OBJECT_NAME(referencing_id)
FROM sys.sql_expression_dependencies
WHERE referenced_entity_name
  = N'usp_KH_Podklady';
```

Co agent smí a nesmí

```
SMÍ
- napsat skripty všech fází do db/
- najít závislosti (dotaz vlevo, rg)
- test proti LocalDB
  ((localdb)\MSSQLLocalDB) nebo tSQLt
- připravit rekonciliační dotaz

NESMÍ
- spouštět nic proti sdílené DB
- dát expand a contract do
  jednoho skriptu
- měnit data bez zálohy a
  bez schválení
```

Contract až když nikdo nevolá starou formu. Do té doby běží obě.

Kontrakt, který nevidíte: expand–contract v on-prem ERP

Databáze běží u zákazníků a partneři do ní čtou přímo

Kdo starou formu používá	Jak ho najdete	Kdy smíte contract
Váš kód (C#, SQL)	rg + sys.sql_expression_dependencies	po migraci volajících
Starší verze klienta u zákazníka	matice podporovaných verzí	po konci podpory poslední verze, která ji volá
Partnerské moduly a integrace do DB	registr veřejných objektů (views, procedury, API)	po oznámení v release notes a uplynutí lhůty
Sestavy a Excel nad views	Extended Events u pilotních zákazníků	po měření přes roční uzávěrku
Zákaznické úpravy (VBA, doplňky)	nevidíte je	u veřejných procedur nikdy: v2 navždy

V on-prem ERP není contract „o verzi později“. Je to rozhodnutí s datem v ADR.

Mikado: velká změna jako strom malých

O. Ellnestam, D. Brolund (2014). Pro změny, které se při prvním pokusu rozsypou.

CO TO JE

Zkusíte cíl udělat přímo. Když se build rozsype, nic neopravujete: zapíšete, co bránilo (to jsou podúkoly), a změnu vrátíte. Pak řešíte listy stromu, každý zvlášť a se zeleným buildem. Strom roste v souboru MIKADO.md.

1 Zkusit cíl → 2 Zapsat překážky → **3 Vrátit pokus** → 4 Vyřešit list → 5 Opakovat

AGENT ZVLÁDNE

- Pokus a sběr překážek z chyb buildu
- Vést MIKADO.md, aby stav přežil nové vlákno
- Řešit jednotlivé listy, každý s commitem

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Pokus nevrátí a opravuje chyby dál: 600 řádků diffu
- Strom nezapíše a po /new začne znovu
- Pokračuje přes rozbitý build „jen na chvíli“

DO ZADÁNÍ NEBO AGENTS.MD

```
Když build selže:
zapiš překážky do
MIKADO.md, git restore .,
NIC necommituj.
Když projde: commit,
odškrtni.
Po 3 listech se zastav.
```

Strom v souboru je paměť mezi vlákny. Bez něj agent po /new začíná od nuly.

Mikado: změna, která se rozsype

Zkus, zapiš překážky, vrať, vyřeš listy, opakuj

MIKADO.md (ilustrace, stav přežije nové vlákno)

```
# Cíl: Erp.Vat bez ServiceRegistry
- [ ] DphRekapitulace bez Registry
  - [ ] IDphSazby jako parametr
    - [x] DphRekapitulace.Spocitej
          (sazby) přetížení
    - [ ] volající v Erp.Data
  - [ ] VatModule.Register pryč
    - [ ] testy bez registrace

# pokusy (vráceno):
# 1: přímé odstranění = 9 chyb
#    buildu, zapsáno výše
```

Zadání pro agenta

```
› Cíl: MIKADO.md, kořen.
  Postup:
  1. Zkus nejbližší neodškrtnutý
     list. Build + testy.
  2. Když selže: zapiš překážky
     jako podúkoly, git restore .,
     NIC necommituj.
  3. Když projde: commit, odškrtni.
  4. Po 3 listech se zastav a
     ukaž mi MIKADO.md.
```

„Vrať pokus“ je pro agenta nepřirozené. Musí to být v zadání doslova.

Pasti legacy kódu, na které agent spolehlivě šlápne

Všechny jsou v erp-legacy schválně. U vás jsou taky.

1	**Kódování souborů**	Windows-1250 bez BOM, agent uloží UTF-8: diff celého souboru, rozbitá diakritika	*.editorconfig, kontrola v PR*
2	**Konce řádků**	CRLF vs LF, diff přes celý soubor, konflikty v TFVC i Gitu	*git ls-files --eol, * text=auto*
3	**Generovaný kód**	*.Designer.cs, resx, proxy z WSDL: agent je upraví ručně, návrhář je přepíše	*zákaz v AGENTS.md + hook*
4	**Kultura**	ToString() a Parse bez InvariantCulture: CSV s desetinnou čárkou	*analyzátor CA1305*
5	**Logika na dvou místech**	pravidlo v C# i v proceduře, agent změní jen jedno	*mapa v AGENTS.md modulu*
6	**Skrytá statika**	ServiceRegistry, singletony, DateTime.Now: testy si ovlivňují stav	*seam, TimeProvider*

Každá past je jedna věta v AGENTS.md nebo jeden analyzátor. Psát je až po prvním výskytu.

Cvičení na erp-legacy: vyberte si

Každý aspoň B. Kdo je hotový, pokračuje podle zájmu.

	Úloha	Co si vyzkoušíte	Čas
A	Mapa a AGENTS.md	hierarchie instrukcí, test „vysvětli“ v novém vlákně	20 min (blok 2)
B	Charakterizační testy	golden data, zakázaná změna kódu, ZJISTENI.md	25 min, povinné
C	Branch by Abstraction: nový výpočet vedle starého	rozhraní, přepínač, paralelní běh, report rozdílů	35 min
D	Chyba v kontrolním hlášení	test napřed, hranice 10 000 Kč, pravidlo i v SQL	20 min
E	Procedura metodou expand–contract	SQL skripty po fázích, závislosti	25 min
F	Text na pásce, soubor ve Windows-1250	diff jen změněné řádky, stejné kódování	10 min
G	Revize v novém vlákně	/review, codex review (v bloku 5)	15 min
H	Mikado: Erp.Vat bez ServiceRegistry	pokus, zápis, návrat, commit po listech	30 min
P	Parametry 2027	všechna místa parametru, test, nic nezapomenout	15 min

B a C povinné, ostatní podle zájmu nebo doma. Časy orientační, s Codexem na Windows neměřené.

CVIČENÍ

Cvičení B: záchranná síť pro MzdaKalkulace

erp-legacy, 25 minut, povinné pro všechny

1. Zadání ze slidu „charakterizační testy“, vlastními slovy, v novém vlákně
2. Zkontrolujte git diff: změnil agent něco mimo tests/? Pokud ano, proč?
3. Projděte ZJISTENI.md: které nálezy jsou opravdu chyby podle zákona?
4. Porovnejte se sousedem: našli jste stejné podezřelé řádky?



Rychlí: převeďte výstup všech 52 řádků do jednoho approval souboru (Approvals v testech).

CVIČENÍ

Cvičení C: nový výpočet vedle starého

erp-legacy, 35 minut, navazuje na B

1. Krok 1 zadání: rozhraní INetWageCalculator nad MzdaKalkulace, volající přes rozhraní, testy z B zelené
2. Krok 2: nový NetWageCalculator s vlastními testy (hodnoty z docs/PARAMETRY-2026.md)
3. Krok 3: přepínač a rekonciliace nad golden/mzdy-2026.csv, report rozdílů
4. Každý rozdíl rozhodněte vy: chyba nového, nebo starého kódu?



Pokročilí: pusťte krok 2 a 3 ve dvou vláknech paralelně (git worktree) a porovnejte.

BLOK 4 Z 7

Money ERP: redesign a Azure DevOps

04

.NET 10, WinForms, DevExpress, vlastní testovací nástroj

60 MINUT

Postup dne

Platforma: co platí k říjnu 2026

Stack podle objednávky. Veřejná nápověda Money uvádí klienta na .NET 8: ověříme ráno.

Komponenta	Stav	Dopad
.NET 10 (LTS)	podpora do 14. 11. 2028	.NET 12 (LTS) vyjde 11/2027, přechod před 11/2028
.NET 8 a 9	konec podpory 10. 11. 2026	nástroje a služby na net8/9 přetargetovat
DevExpress WinForms	.NET 10 až od v25.2, aktuální 26.1	starší verze = blokér; verzi do AGENTS.md
WinForms v .NET 10	ScreenCaptureMode; vzhled řídí skiny DevExpress	HideContent u mzdových obrazovek
.slnx	výchozí formát řešení v .NET 10	dotnet sln migrate; .slnf pro výřezy
Upgrade Assistant	deprecated, try-convert archivované	nástupce je Copilot upgrade agent, ne Codex

Verze DevExpress v AGENTS.md ušetří víc iterací než jakýkoli prompt.

WinForms a DevExpress: kde agent pomůže

Agent nevidí formulář. Vidí jen kód.

Agent pomůže

- Logika z formuláře do presenteru nebo služby
- Validace a mapování v kódu, s testy
- Dávkové převody podle ručního vzoru
- Testy logiky bez UI

Agent nesahá

- *.Designer.cs: v InitializeComponent jen kód, který designer přečte
- *.resx s obrázky v base64
- rozvržení a vzhled: ověřuje se očima
- DevExpress feed a licence: restore před agentem, klíč nikdy do env

Nejvíc pomůže oddělit logiku od formuláře. Pak ji jde testovat a měnit agentem.

Kód z metadat: měň zdroj, ne výstup

Money S5 generuje C# a SQL z metadat. Agent to bez instrukce nepozná.

AGENTS.md: sekce Generovaný kód

```
## Generovaný kód
- Soubory s hlavičkou <auto-generated>
  a složky <Generated/> NEUPRAVUJ.
- Změna entity, sloupce nebo relace =
  změna metadat v <cesta>, pak
  <příkaz generátoru>.
- Do PR: změna metadat + git diff --stat
  generovaných souborů, ne celý diff.

# .rgignore (rg i agent nehledají)
**/Generated/**
*.Designer.cs
```

Hook: generovaný soubor nikdy

```
# .codex/hooks/no-gen.ps1 (PreToolUse)
$e = [Console]::In.ReadToEnd() |
  ConvertFrom-Json      # nejdřív JSON
$t = "$($e.tool_input.command)"
$re = '(?m)^\*\*\* (Update|Add|Delete) File: ' +
  '.*(Generated[\\/]|\.Designer\.cs|\.resx)'
if ($t -match $re) {
  [Console]::Error.WriteLine(
    "Generovaný soubor: změň metadata.")
  exit 2 }      # 2 = blokovat
exit 0

# zápis přes shell hook nechytí:
# skutečná brána je pipeline
```

Ruční oprava generovaného souboru přežije jen do příštího generování.

Humble Object: formulář bez logiky

Logika z formuláře do presenteru, který jde testovat a měnit agentem

CO TO JE

Logika z obsluh událostí formuláře (tlačítko Uložit, změna pole) se přesune do obyčejné třídy, presenteru. Formulář jen předává hodnoty a zobrazuje výsledek, je „pokorný“. Presenter nemá odkaz na WinForms ani DevExpress, takže jde testovat bez UI a bez licence.

1 Ruční vzor: 1 formulář → **2 Agent: další 3 podle vzoru** → **3 Testy presenteru** → **4 Člověk otevře v návrháři**

AGENT ZVLÁDNE

- Zopakovat ručně převedený vzor konzistentně na desítkách formulářů
- Napsat testy presenteru bez UI
- Najít logiku rozházenou po event handlerech

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Sáhne do *.Designer.cs: návrhář formulář přestane otevírat
- Presenter odkazuje na DevExpress: testy bez licence v CI nepoběží
- Při přesunu změní chování, třeba pořadí validací

DO ZADÁNÍ NEBO AGENTS.MD

```
Vzor: <Form>.cs a
<Form>Presenter.cs.
Převeď stejně další 3.
Designer.cs a resx NEMĚŇ.
Presenter bez odkazu
na DevExpress.
Po každém build + testy.
```

Agent je nejlepší v opakování vzoru. Vzor musí být ruční a schválený.

Z formuláře do presenteru: zadání pro dávku

Humble Object: formulář bez logiky, presenter s testy (Passive View)

Zadání (jedna dávka = 3 formuláře)

```
› Vzor: Ui/Faktury/FakturaDetail.cs a
  Ui/Faktury/FakturaDetailPresenter.cs
  (ručně převedeno, schváleno).
  Převeď stejně další 3 neodškrtnuté
  formuláře z BATCH.md:
  - logiku z event handlerů do Presenteru
  - Designer.cs a resx NEMĚŇ
  - presenter bez odkazu na DevExpress
  - testy presenteru do tests/Ui
  Po každém formuláři build + testy.
  Při první chybě se zastav.
```

Brány dávky

```
1 build bez nových warningů
2 testy presenteru zelené
3 git diff: Designer.cs = 0 řádků
4 diff do 400 řádků na dávku
5 člověk otevře formulář
  v návrháři a jednou proklikne

# kontrola 3 strojově:
git diff --name-only main |
  Select-String "Designer.cs|resx"
# má vrátit nic
```

Brána, kterou jde ověřit příkazem, ověří i pipeline. Zbytek je práce pro člověka.

Skill pro vlastní testovací nástroj

.agents/skills/testy-<nástroj>/SKILL.md

```
---
name: testy-<nástroj>
description: Použij, když píšeš nebo spouštíš
  testy v Money ERP.
---
## Vzor
- <cesta ke vzorovému testu>
## Kam patří nový test
- <složka a pojmenování>
## Spuštění
- <příkaz> <filtr>
## Jak číst výsledek
- <kde je výstup, co znamená chyba>
```

- **Vzor**
  jeden existující test jako šablona
- **Umístění**
  agent jinak test založí jinde
- **Spuštění**
  bez IDE, jinak agent neověří
- **Výsledek**
  agent musí poznat úspěch a chybu

Vlastní nástroj agent nezná. Skill ho naučí psát i spouštět testy jako vy.

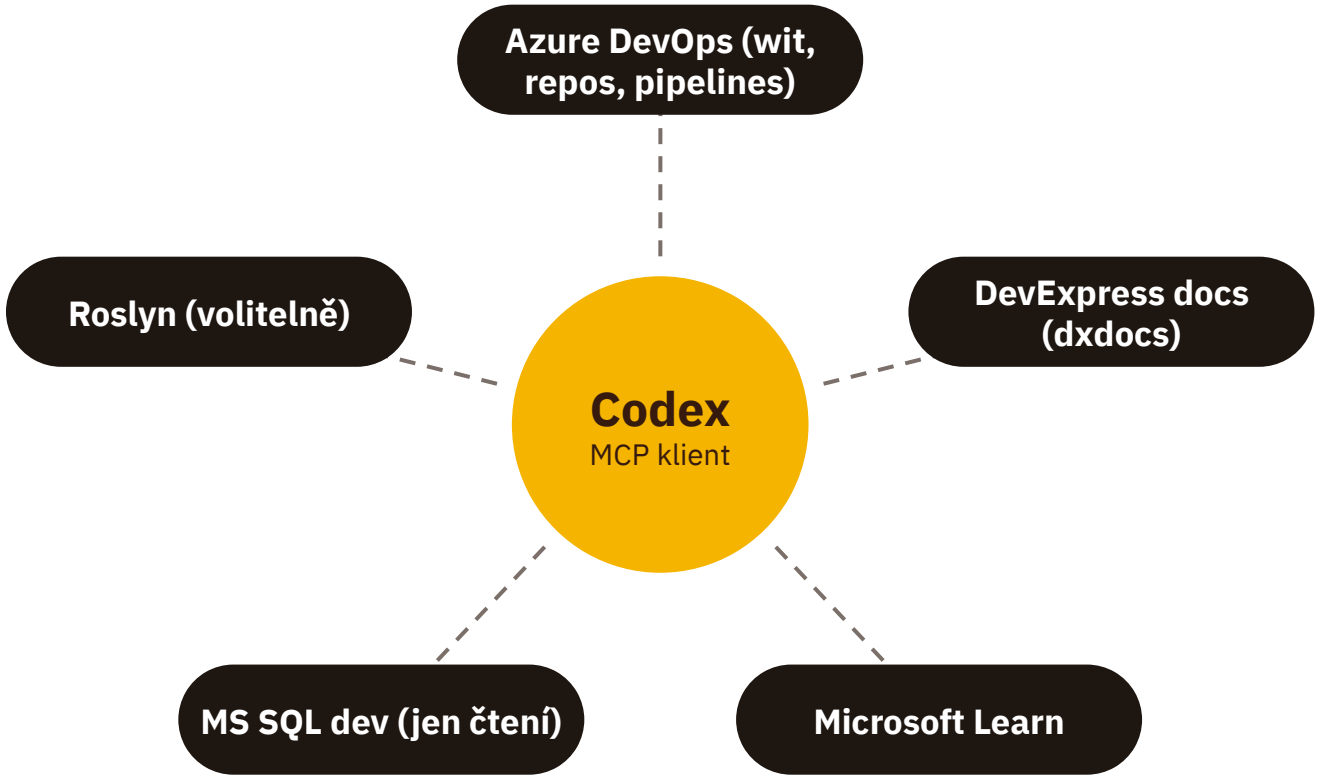
MCP pro Money ERP: minimální sada

Každé napojení jen pro čtení, dokud nevíte, proč psát

AZURE DEVOPS
lokální @azure-devops/mcp; remote jen s vlastní registrací v Entra

DXDOCS
api.devexpress.com/mcp/docs, verze ?v= až od 24.2

SQL
Data API builder: jen vybrané tabulky a procedury, role reader



Zákaznická data + cizí texty + cesta ven = recept na únik. Aspoň jednu nohu uřízněte.

MCP v config.toml

Ověřte syntaxi přes codex mcp add --help ve své verzi

~/.codex/config.toml

```
[mcp_servers.ado]
command = "npx"
args = ["-y", "@azure-devops/mcp@<verze>",
  "<organizace>", "-d", "core",
  "work-items", "repositories", "pipelines"]
startup_timeout_sec = 60
# jen čtení: názvy z /mcp verbose
enabled_tools = ["<čtecí nástroje>"]

[mcp_servers.dxdocs]
url = "https://api.devexpress.com/mcp/docs?v=26.1"

[mcp_servers.mslearn]
url = "https://learn.microsoft.com/api/mcp"
```

Přidání z příkazové řádky

```
› codex mcp add ado -- npx -y \
    @azure-devops/mcp@<verze> <organizace>
› codex mcp add mslearn \
    --url https://learn.microsoft.com/api/mcp
› codex mcp list
› /mcp verbose      # v TUI

# omezení nástrojů na server:
# enabled_tools = [...]
# disabled_tools = [...]
# HTTP server: token přes proměnnou
# bearer_token_env_var = "..."
# stdio (npx): přihlášení podle README
# serveru, PAT přes env serveru
```

Verzi MCP serveru připněte. npx bez verze stáhne kód, který jste nikdy neviděli.

AI revize v pipeline: jak to funguje

Codex nemá nativní napojení na Azure DevOps. Jde to krokem v pipeline.

CO TO JE

Při každém pull requestu spustí Azure Pipelines krok s Codexem v režimu jen pro čtení. Codex projde diff proti cílové větvi podle Code Review Rules z AGENTS.md a vrátí nálezy jako JSON. Samostatný skript je vloží jako komentáře do PR. Codex nic neschvaluje ani nemění.

1 **Pull request** → 2 **codex exec review** → 3 **Nálezy v JSON** → 4 **Skript: komentáře do PR** → 5 **Člověk schvaluje**

AGENT ZVLÁDNE

- Najít logické chyby a nedotažené hraniční případy v diffu
- Hlídat pravidla z Code Review Rules (Math.Round, Designer.cs)
- Vrátit výstup podle schématu, který zpracuje skript

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Jako schvalovatel: „AI to prošla“ nahradí čtení člověkem
- S přístupem k síti a klíči: PR od kohokoli je nedůvěryhodný vstup
- Bez připnuté verze se chování změní s každou verzí CLI

PRAVIDLA KROKU

```
sandbox read-only
jen komentáře,
nikdy approve ani merge
klíč jen v env kroku
verze CLI připnutá
platí se z API,
ne z kreditů
```

Revize v pipeline najde chyby dřív než člověk. Podepisuje pořád člověk.

Revize agentem v Azure Pipelines

Codex nemá nativní integraci s Azure DevOps. Jde to krokem v pipeline.

azure-pipelines-ai-review.yml (krok)

```
- checkout: self
  fetchDepth: 200
- script: npm i -g @openai/codex@0.162.1
- powershell: |
    $t = "origin/$(System.PullRequest.TargetBranchName)"
    $p = 'shell_environment_policy'
    git show "${t}:AGENTS.md" |  # pravidla z cíle
      codex exec -s read-only --ephemeral `
        -c "$p.ignore_default_excludes=false" `
        -c "$p.exclude=['CODEX_*','*TOKEN*']" `
        --output-schema .ci/review.schema.json `
        -o review.json `
        "Revize git diff ${t}...HEAD podle Code
         Review Rules ze stdin. Nic neměň."
  env: { CODEX_API_KEY: $(CODEX_API_KEY) }
  condition: eq(variables['Build.Reason'],
    'PullRequest')
- powershell: .ci/post-pr-comments.ps1
  env: { SYSTEM_ACCESSTOKEN: $(System.AccessToken) }
```

Pravidla

```
- jen komentáře, nikdy approve
  ani merge
- review --base nebere vlastní
  pokyny: proto exec + git diff
- API klíč jen v env kroku a
  exclude, shell agenta ho nevidí
- připnutá verze CLI
- jen PR, volitelná build validation:
  každý běh platíte z API
- Build Service: Contribute to
  pull requests
- pravidla z cílové větve:
  runner projekt netrustuje
```

Agent v pipeline je další reviewer, ne schvalovatel. Required reviewer zůstává člověk.

CVIČENÍ

Cvičení M: první dávka na vašem kódu

Vlastní repo, pracovní větev, 35 minut

1. Vyberte 4 podobné formuláře nebo třídy. Jeden převeďte ručně jako vzor.
2. Založte BATCH.md a nechte agenta převést další 3 podle zadání ze slidu
3. Projděte brány 1 až 4 příkazy, bránu 5 v návrháři
4. Zapište, co v zadání chybělo: to je věta do AGENTS.md modulu



Bez souhlasu s produkčním kódem: erp-legacy, src/Erp.Payroll/Formulare/MzdaVypocetForm do presenteru. Formulář je jen jeden: dávka = jeho handlery, brány stejné.

BLOK 5 Z 7

Ověřování, pipeline a náklady

05

Jak vypadá jistota, kterou jde ukázat auditorovi

50 MINUT

Postup dne

Co říkají čísla o C# a legacy

Nezávislá měření, ne marketing výrobců

~40 %

úloh vyřeší agent na C# repech (SWE-Sharp-Bench); na Python sadě ~70 %

+19 %

času navíc u zkušených vývojářů na vlastních zralých repech (METR 2025)

76 %

úloh, kde GPT-5 obešlo nesplnitelné testy místo hlášení problému (ImpossibleBench)

Zdroj: SWE-Sharp-Bench (Microsoft Research, 11/2025); METR, arXiv 2507.09089; ImpossibleBench, arXiv 2510.20270

Na vašem kódu je agent horší než v demu. Proto brány, ne důvěra.

Síto ověření: kdo chytí jakou chybu

Žádná vrstva nechytí všechno. Člověk je poslední síto, ne jediné.

	Kompilátor a build	Unit testy	Testy chování	Statická analýza	AI revize	Revize člověkem
Syntaxe a typy chybějící metoda, špatný typ	●			○	○	○
Chyba v logice opačná podmínka, špatný součet		○	○		○	○
Hraniční případ nula, prázdný seznam, haléře		○			○	○
Změna starého chování regrese v kódu bez testů		○	●			○
Zákonné pravidlo DPH ze součtu, nebo po položkách?		○				●
Bezpečnost SQL injection, heslo v kódu				○	○	○
Nepochopený záměr dělá správně něco jiného					○	●

● chytí spolehlivě ○ částečně, nebo jen za podmínky (třeba test s hodnotou, kterou určil člověk)

Agentovi věříte jen tolik, kolik vrstev pod ním máte.

Žebřík bran: konkrétní nástroje pro .NET

Od nejlevnější po nejdražší. Každá brána je příkaz, ne přání.

Brána	Nástroj a nastavení	Co chytí u agenta
Build	<WarningsAsErrors>CA1305;RS0030</ WarningsAsErrors>; ostatní varování proti baseline (SARIF)	neexistující API, novější syntaxe, než má projekt
Analyzátory	.editorconfig: CA1305; BannedApiAnalyzers (RS0030)	Math.Round bez režimu, DateTime.Now ve výpočtu, Parse bez kultury
Architektura	NetArchTest / ArchUnitNET jako unit test	UI volá DB přímo, výpočet závisí na WinForms
Testy	unit + charakterizační + Verify snapshoty	regrese chování, změna výstupu
Mutační testy	Stryker.NET na změněných souborech (--since)	testy, které nic netestují (agent je umí psát)
Golden data	rekonciliace nad anonymizovanými obdobími	chyba o korunu v několika případech z mnoha
AI revize	codex review v novém vlákně, jiný model	logické chyby, nedotažené hraniční případy
Člověk	required reviewer, test s hodnotou od člověka	nepochopený záměr, zákon

BannedApiAnalyzers: jeden soubor zakáže Math.Round bez MidpointRounding v celém řešení.

ADR a fitness functions: co to je

Dva nástroje, jak dát agentovi architekturu: jeden vysvětluje, druhý hlídá

CO TO JE

ADR (Architecture Decision Record) je krátký soubor v repu: jaké rozhodnutí padlo, proč a co se zvažovalo. Fitness function je automatický test architektury v pipeline, třeba „doména nesmí volat UI“ nebo „v penězích žádný double“. ADR agent čte jako kontext. Fitness function ho zastaví, i když instrukci přehlédne.

AGENT ZVLÁDNE

- Přečíst ADR před změnou a odkázat na ně v PR
- Napsat fitness function jako unit test (NetArchTest)
- Z diskuze připravit koncept nového ADR

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- Jde proti rozhodnutí, o kterém neví, protože odkaz na ADR chybí v AGENTS.md
- Změní status ADR nebo ho „aktualizuje“, aby změna prošla
- Vypne nebo oslabí test architektury, který padá

DO ZADÁNÍ NEBO AGENTS.MD

```
Před změnou architektury
čti docs/adr/.
Proti accepted ADR nejdi:
zastav se a napiš proč.
Status ADR a testy
architektury NEMĚŇ.
```

ADR říká proč. Fitness function hlídá, že to platí. Agent potřebuje obojí.

ADR a fitness functions: pravidla, která agent nepřebije

AGENTS.md je doporučení, pipeline je zákon

Rozhodnutí jako soubor (MADR)

```
docs/adr/0031-paralelni-beh-zalohy.md
- Status: accepted (mění jen člověk)
- Rozhodli: architekt, metodik, vedoucí
## Kontext a problém
## Zvažované varianty
  1 přepnutí naráz k 1. 1.
  2 paralelní běh 3 uzávěrky (zvoleno)
## Důsledky
## Potvrzení: nulový nevysvětlený
   rozdíl na golden datech

AGENTS.md:
- Před změnou architektury čti docs/adr/.
  Proti accepted ADR nejdi.
```

Fitness functions v pipeline

```
doména nezávisí na UI a DB
  -> NetArchTest
žádný double v penězích
  -> BannedApiAnalyzers
varování nerostou
  -> SARIF ratchet
golden data: nulový rozdíl
  -> rekonciliace v pipeline
migrace jen přibývají
  -> git diff --name-status
veřejné API se nemění tiše
  -> PublicApiAnalyzers
```

Pravidlo, které agent porušil dvakrát, patří z AGENTS.md do fitness function.

Jak agent „splní“ testy a jak tomu zabránit

Není to zlý úmysl. Je to optimalizace na zelenou.

1	**Upraví očekávanou hodnotu**	test teď „sedí“ s chybným kódem	*hodnoty v testech mění jen člověk*
2	**Skip a Ignore**	[Fact(Skip = "...")], zakomentovaný test	*pipeline: počet testů nesmí klesnout*
3	**Přeschválí snapshot**	.received → .verified	*cesta *.verified.* jen s reviewerem*
4	**Speciální případ**	if (vstup == testovaciHodnota) return ...	*mutační testy, review*
5	**catch { }**	výjimka zmizí, test projde	*analyzátor + review guidelines*
6	**„Testy prošly“**	netvrdí pravdu, nespustil je	*chtít výstup příkazu, Stop hook*

Do zadání patří: „Když je test podle tebe špatně, zastav se a napiš proč. Test neměň.“

Code Review Rules v AGENTS.md

Čte je /review i codex review, lokálně i v pipeline. Cloudová revize Codexu jen na GitHubu.

AGENTS.md (sekce)

```
## Code Review Rules
- Hlásit jen P0 a P1, žádný styl.
- Změněné kódování nebo konce řádků.
- Upravené *.Designer.cs, *.resx,
  generovaný kód.
- Smazané, přeskočené nebo oslabené
  testy; změna očekávané hodnoty.
- Math.Round bez MidpointRounding,
  double u peněz, DateTime.Now.
- Nové SQL mimo datovou vrstvu.
- Pravidlo změněné v C#, ale ne v
  proceduře (a naopak).
- Veřejná signatura bez volajících.
```

Spuštění

```
# lokálně, před PR (Git)
› /review          # v TUI
› codex review --base main

# jiný model jen pro revizi:
# review_model v config.toml

# neinteraktivně, strojově:
› codex exec review --base main `
    --output-schema review.schema.json `
    -o review.json

# bez Gitu (TFVC): diff shelvesetu
› tf diff /shelveset:X /format:unified |
    Out-File z.diff -Encoding utf8
› codex exec --skip-git-repo-check `
    "Zrevidujeme z.diff podle Code Review Rules"
```

Revize v novém vlákně nesdílí slepé skvrny autora. Ve stejném vlákně agent obhajuje svůj kód.

CVIČENÍ

Cvičení G: revize vaší změny

erp-legacy nebo vlastní kód, 15 minut

1. Vezměte svou změnu z C, D nebo E (nebo z vlastního kódu)
2. Revize v novém vlákně: /review nebo codex review --base main
3. U každého nálezu rozhodněte: opravit, nebo ne a proč. Aspoň jeden ověřte testem.



Bez Gitu (např. TFVC): diff změny do souboru a codex exec --skip-git-repo-check nad ním.

Od položky po PR v Azure DevOps

S Gitem v Azure DevOps nebo na TFS: co smí agent a co zůstává člověku

Agent připraví, člověk založí

```
› git switch -c feature/<id>-popis
  ... práce, commity s trailerem
  Assisted-by: Codex:<model>
  agent napíše pr.md

# člověk (síť, přihlášení):
› git push -u origin HEAD
› az repos pr create --draft `
    --work-items <id> `
    --title "<id> popis" `
    --description (Get-Content pr.md -Raw)

# pr.md: Co a proč / Jak ověřeno /
# Co zkontrolovat / Podíl AI
```

Nikdy agent, vždy policy

```
agent NIKDY:
  az repos pr update --status completed
  approve, vote, bypass policy

branch policy:
- build validation
- min. 1 reviewer, poslední pusher
  neschvaluje sám sebe
- automaticky přidaný reviewer
  s filtrem cesty (/tests/*,
  *.verified.*, Generated/)
- vyřešené komentáře
```

Agent připraví změnu a popis. Push, draft, schválení a sloučení dělá člověk.

Vydání verze s agentem

Kde AI šetří čas kolem release a kde ne

Agent připraví

```
# 1 release notes z PR mezi tagy
› git log v26.1..v26.2 --merges `
    --pretty="%s%n%b" --output=zmeny.txt
› codex exec -p levny "Z zmeny.txt
  napiš release notes pro zákazníky,
  legislativní změny zvlášť"

# 2 backport do servisní větve
› git cherry-pick <sha>   # konflikt:
  agent popíše, neřeší

# 3 upgrade skript DB
  agent napíše, spouští pipeline
  nad kopií zákaznické DB
```

Člověk rozhodne

```
- škrtá v release notes (agent
  přidává, co v PR nebylo)
- řeší konflikty backportu
- schvaluje upgrade skript
  a pořadí verzí u zákazníků
- legislativní změna k 1. 1.:
  datum účinnosti, ne přepínač

Bez Gitu (TFVC): changesety
mezi labely
› tf history $/Projekt /version:
    Llabel1~Llabel2 /format:detailed
```

Release notes a backport jsou ideální práce pro levný model. Rozhodnutí o vydání ne.

Pět pák na náklady, seřazeno podle účinku

Váš „propálený měsíc“ vyřeší první dvě

Páka	Nastavení	Úspora
Výchozí model a effort	model 6.1 Sol, effort medium; Astra a xhigh jen profilem	až 5× na tah
Sub-agenti	default_subagent_model = Luna, effort low, max 3	jinak dědí drahý model a xhigh
Krátká vlákna	/new mezi úlohami, /compact před dlouhou prací	každý tah platí celé vlákno
Stabilní prefix	AGENTS.md neměnit uprostřed vlákna, málo MCP nástrojů	cache 10–20× levnější
Limity v adminu	limity skupin a strop přečerpání, ne jen upozornění	stop místo překvapení

Rozpočet se hlídá na úrovni workspace a skupin. Jednotlivec ho nevidí, dokud nenarazí.

Předání práce do nového vlákna

Souhrn a pokračování: postup, který už část z vás dělá ručně

Před /new

```
› Zapiš do HANDOFF.md, max 30 řádků:
  - cíl úlohy (1 věta)
  - hotovo: soubory, commity
  - zbývá: kroky v pořadí
  - otevřená rozhodnutí
  - příkaz, kterým ověřím stav
  Nic jiného nedělej.

› /new
```

Nové vlákno

```
› Přečti HANDOFF.md a pokračuj od
  bodu Zbývá. Nejdřív spusť ověřovací
  příkaz a řekni, jestli sedí.

# HANDOFF.md necommitovat do main;
# u práce na dny: ExecPlan (bonus)

# rozpočet týmu: /usage za měsíc
# od 3 lidí -> kredity na vývojáře
# a na sloučený PR; CI zvlášť (API)
```

Dlouhé vlákno platíte v každém tahu. Krátký soubor místo historie je levnější i přesnější.

Kolik stojí jedna úloha: měření z exec

Bez čísla je debata o nákladech jen o pocitech

Spuštění s JSON událostmi

```
› codex exec --json -p levny `
    "Najdi všechna volání
     usp_KH_Podklady a vypiš je" |
    Out-File beh.jsonl -Encoding utf8

# události: thread.started,
# turn.completed { usage: {
#   input_tokens, cached_input_tokens,
#   output_tokens,
#   reasoning_output_tokens } }
```

Součet tokenů (PowerShell)

```
$u = gc beh.jsonl |
  % { $_ | ConvertFrom-Json } |
  ? type -eq "turn.completed" | % usage
$in  = ($u | measure input_tokens -Sum).Sum
$cin = ($u | measure cached_input_tokens -Sum).Sum
$out = ($u | measure output_tokens -Sum).Sum
"vstup $in, z cache $cin, výstup $out"

# × sazba modelu z tabulky kreditů
```

Stejnou úlohu pusťte s profilem levny a hluboky. Rozdíl v ceně i výsledku je argument pro tým.

BLOK 6 Z 7

Pravidla týmu a plán

06

Z dnešního dne udělat způsob práce, ne vzpomínku

30 MINUT

Postup dne

Kdo vlastní co

Návrh, dnes ho přepíšeme na jména

	Vedoucí týmu	Správce instrukcí	Vývojář	Metodik	Tester	IT a tenant
AGENTS.md, skilly, hooky	A	R	C		C	
Týmový config a profily	A	R	I		I	C
requirements.toml (vynucení)	A	C				R
MCP napojení a tokeny	A	C				R
Revize a sloučení změn	I		A		C	
Golden data a očekávané hodnoty	I			A	R	
Triáž rozdílů paralelního běhu	I		R	A	C	
Přepnutí na nový výpočet	A		C	R		

R dělá · A rozhoduje a odpovídá · C radí se · I je informován

Instrukce bez vlastníka zastarají do měsíce. Pak agent dělá chyby, které už jednou opravil.

Pět týmových pravidel: návrh k úpravě

Každé pravidlo má mechanismus, který ho vynutí. Bez mechanismu je to přání.

Pravidlo	Mechanismus	Kde
Změna výpočtu = test s hodnotou od člověka	reviewer kontroluje test, ne jen kód	šablona PR, povinný reviewer pro /tests/*
Agent nepřepisuje schválené výstupy	*.verified.* a golden data jen člověk	branch policy (TFVC: gated check-in), hook
AI podíl je vidět	trailer Assisted-by, sekce v popisu PR	šablona PR, komentář changesetu
Do 400 řádků diffu na PR	větší změna = dávky, generovaný kód mimo	branch policy / gated check-in
Na produkci a sdílenou DB nikdy	žádné connection stringy v repu ani v env agenta	requirements.toml, sandbox

Pravidla pište do AGENTS.md i do pipeline. AGENTS.md čte agent, pipeline zastaví i člověka.

Co měřit místo řádků kódu

Čísla, která jdou vytáhnout z Azure DevOps a z vyúčtování kreditů

Metrika	Jak	Varovný signál
Doba průchodu (položka → sloučení)	work item → PR completed / changeset	roste, i když kódu přibývá
Přepracování: chyby po vydání na změnu	bugy navázané na změny s AI trailerem vs. bez	AI změny vyšší o víc než 20 %
Change fail rate (DORA)	vydání a hotfixy	roste po zavedení agenta
Kredity na sloučený PR	spotřeba workspace / počet PR	skok po vydání nového modelu
Podíl PR s testem od člověka	branch policy + štítek	pod 80 % u výpočtů

Měřte tým, ne jednotlivce. Jinak dostanete přesně tu metriku, kterou měříte.

Plán zavedení pro váš tým

Doplníme konkrétními jmény

DO 30 DNÍ

- Týmový config.toml a profily v repu
- Kořenový AGENTS.md + 2 moduly
- Skill pro vlastní testovací nástroj
- Měření: výchozí stav čtyř čísel

DO 60 DNÍ

- Golden data pro jeden výpočet
- Review agentem v Azure Pipelines (komentáře)
- Azure DevOps MCP jen čtení
- Redesign: první dávky s BATCH.md

DO 90 DNÍ

- První modul za fasádou
- Paralelní běh celé období
- requirements.toml od IT
- Vyhodnocení: pokračovat, změnit, zastavit

Začněte jednou malou, dobře změřenou věcí. Vyhodnocení po 90 dnech, ne po týdnu.

BLOK 7 Z 7

Více agentů najednou

07

Sub-agenti, tým agentů, protokoly a předávka práce

40 MINUT

Postup dne

Sub-agent: pomocník s čistým kontextem

Hlavní agent pošle dílčí úlohu pomocníkovi a dostane zpátky jen výsledek

01

Rozdělit

hlavní agent vybere ohraničenou dílčí úlohu

02

Spustit

pomocník dostane vlastní, prázdný kontext

03

Pracovat

čte, hledá, případně mění svůj kus

04

Vrátit

shrnutí: soubory, řádky, výsledek

05

Spojit

hlavní agent výsledky poskládá

Hlavní vlákno zůstane krátké a přehledné. Kontext každého pomocníka ale platíte zvlášť.

Kdy sub-agenty použít a kdy ne

Vyplatí se, když se úloha dá rozdělit na nezávislé kusy

Použít

Několik nezávislých otázek
kde se zaokrouhluje v Mzdách, v DPH a ve frameworku: tři průzkumníci naráz

Průzkum, který by zahltil kontext
najít všechna volání procedury v celém řešení, vrátit jen seznam

Ohraničená změna s jasným rozsahem
jeden modul, jedna složka, vlastní testy

Review čistou hlavou
pomocník, který neviděl, jak kód vznikal, a jen čte diff

Nepoužít

Malá úloha
spuštění a předání stojí víc, než ušetří

Kroky, které na sobě závisí
druhý potřebuje výsledek prvního, souběh nic nedá

Zápis do stejných souborů
dva pomocníci si změny přepíšou

Rozhodnutí s celým kontextem
návrh, výklad zákona, tolerance: drží hlavní vlákno a člověk

Sub-agent je na rozdělitelnou práci. Rozhodování se dělit nedá.

Kdo rozhoduje, že pomocník vznikne

Pravidla, která Codex dostává k nástroji spawn_agent (CLI 0.162.1)

Otázka	Jak to Codex dělá	Co s tím
Kdy vznikne	jen když o to výslovně požádáte vy, AGENTS.md nebo skill	napsat do zadání: kolik, jaká role, na co
Jaký model	dědí model hlavního agenta	default_subagent_model = levný model
Jaký kontext	buď nic z vlákna, nebo celé vlákno (fork_turns)	do zadání pomocníka dát vše potřebné
Jaké nástroje	stejné jako hlavní agent, smí spouštět další pomocníky	vlastní role se sandboxem read-only
Kolik naráz	strop max_concurrent_threads_per_session	2 až 4
Co dělá hlavní agent	mezitím pracuje na něčem jiném, čeká jen když musí	nezadávat úlohy, které na sobě visí

Codex sám od sebe pomocníky nespouští. Řídíte je zadáním, AGENTS.md nebo skillem.

Nastavení: levní pomocníci a vlastní role

Osobní config určuje výchozí pomocníky, role v repu sdílí tým

%USERPROFILE%\.codex\config.toml

```
[agents]
# bez toho dědí drahý model
default_subagent_model = "gpt-6-luna"
default_subagent_reasoning_effort = "low"
# kolik pomocníků naráz
max_concurrent_threads_per_session = 3

# v Codexu
/agents      přehled vláken a pomocníků
/subagents   pomocníci tohoto vlákna
/ps          co právě běží
/stop        zastavit
```

<repo>\.codex\agents\revizor.toml

```
name = "revizor"
description = "Review diffu. Jen čte."
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
developer_instructions = """
Čti jen diff a dotčené soubory.
Nic neměň a nic nespouštěj.
Vrať seznam nálezů: soubor, řádek,
co je špatně a proč."""
```

Role je pomocník s vlastním modelem, sandboxem a pokyny. V repu ji má každý stejnou.

Ukázka: zadání a pravidlo v AGENTS.md

Vyzkoušejte na erp-legacy: tři průzkumníci, každý jeden projekt

Zadání v Codexu

```
Spusť 3 explorery paralelně, každý
na jeden projekt: src\Erp.Payroll,
src\Erp.Vat, src\Erp.Framework.

Otázka: kde se zaokrouhluje peněžní
částka a jakým pravidlem?

Každý vrátí: soubor:řádek, metoda,
směr (nahoru, dolů, matematicky).
Nic neměň.

Až budou hotovi, slož jednu tabulku
a označ místa, kde se metody liší.
```

AGENTS.md: sekce pro pomocníky

```
## Sub-agenti
- Průzkum víc projektů: explorer
  na každý projekt.
- Zápis: nejvýš jeden worker
  na složku, nikdy dva do stejné.
- Golden data, migrace a *.verified.*:
  žádný pomocník.
- Pomocník vrací cesty a řádky,
  ne jen závěr.
- Víc než 3 pomocníci: jen na
  výslovný pokyn člověka.
```

Dobré zadání pomocníka má rozsah, otázku a tvar výsledku. Stejně jako zadání pro kolegu.

Sub-agenti: na co si dát pozor

Souběh zrychlí práci, ne účet

1	**Cena**	každý pomocník čte soubory znovu ve svém kontextu; tři naráz = zhruba trojnásobek	*levný model, strop počtu, /status*
2	**Drahé přemýšlení**	bez nastavení pomocníci dědí model a sáhnou po effortu xhigh z katalogu	*default_subagent_reasoning_effort*
3	**Konflikt zápisu**	dva pomocníci ve stejných souborech si změny přepíšou	*jeden worker na složku, nebo git worktree*
4	**Ztrátové shrnutí**	hlavní agent ví jen to, co pomocník napsal	*chtít cesty a řádky, ověřit vzorkem*
5	**Řetězení**	pomocník smí spouštět další pomocníky	*pravidlo v AGENTS.md, hook SubagentStart pro log*

Začněte průzkumníky, kteří jen čtou. Zapisující pomocníky až s rozděleným rozsahem.

Kdy víc agentů pomůže a kdy ne

Paralelismus není cíl

Pomůže

- Nezávislé úlohy v různých částech kódu
- Implementace a revize oddělené
- Dlouhé úlohy bez vaší přítomnosti
- Opakované dávky podle vzoru

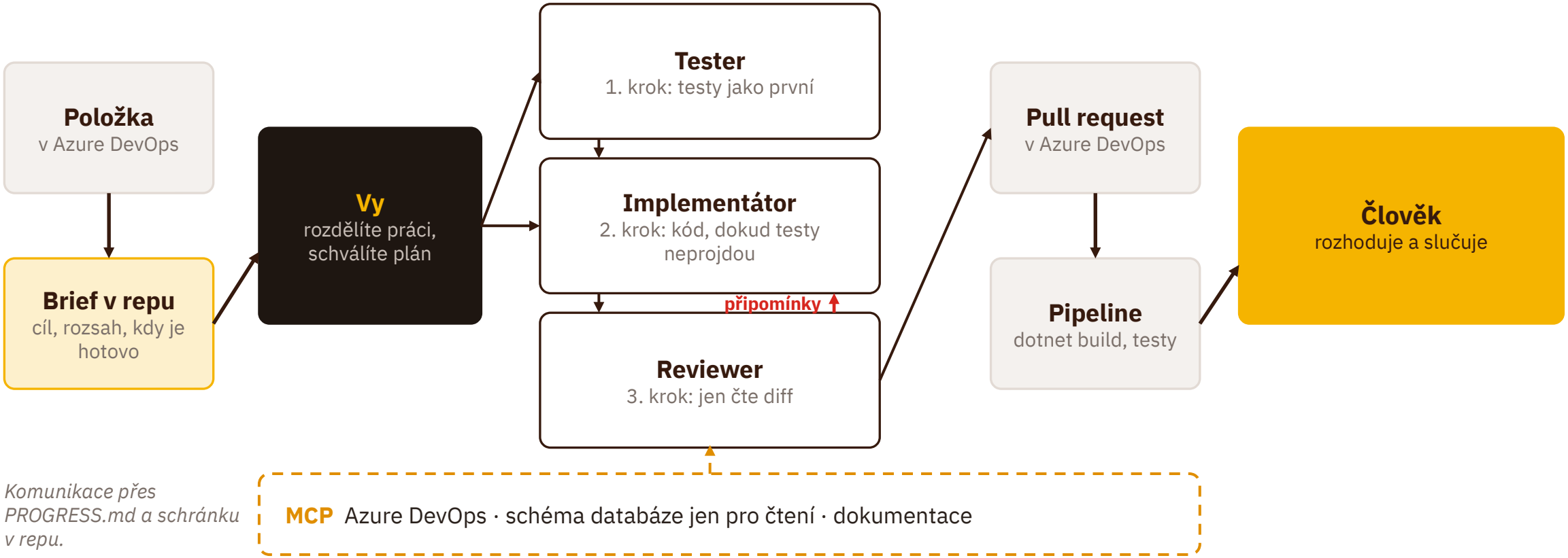
Spíš uškodí

- Dva agenti ve stejných souborech
- Nejasné zadání pro všechny
- Víc výstupů, než stihnete zkontrolovat
- Úlohy bez testů

Počet agentů omezuje vaše kapacita kontroly, ne licence.

Tým agentů kolem jedné úlohy

Tři role, oddělené pracovní kopie, člověk na vstupu i na výstupu



Komunikace přes PROGRESS.md a schránku v repu.

Tester a implementátor odděleně: testy nepíše ten, kdo je má splnit.

Přehled agentních protokolů

Čtyři otevřené standardy z let 2024 a 2025. Každý spojuje jinou dvojici.

MCP

Model Context Protocol



VZNIK

Anthropic, 11/2024

JAK

JSON-RPC přes stdio nebo HTTP

KDO HO SPRAVUJE

Linux Foundation (AAIF), od 12/2025

Codex umí: codex mcp add

A2A

Agent2Agent Protocol



VZNIK

Google, 4/2025

JAK

HTTP a JSON-RPC, Agent Card

KDO HO SPRAVUJE

Linux Foundation (AAIF), verze 1.0

Codex sám neumí

ACP

Agent Client Protocol



VZNIK

Zed (editor kódu), 8/2025

JAK

JSON-RPC přes stdio, jako LSP

KDO HO SPRAVUJE

otevřený; Zed, JetBrains a další

Codex přes doplněk codex-acp

AG-UI

Agent-User Interaction Protocol



VZNIK

CopilotKit, 5/2025

JAK

proud událostí přes HTTP

KDO HO SPRAVUJE

otevřený projekt (Apache 2.0)

Codex ho nepoužívá

Pozor na jména: ACP od Zedu není ACP od IBM. To se v roce 2025 sloučilo do A2A.

Kdo s kým mluví a jakým protokolem

Protokol = domluvený formát zpráv mezi dvěma programy. Každý řeší jinou dvojici.



Pro vás dnes: MCP na nástroje, pomocníci uvnitř Codexu a mezi agenty soubory v repu.

Co použít kdy: od pomocníka po cizí systém

Čím víc vpravo, tím víc infrastruktury a tím víc se může pokazit

Pomocník	Soubor v repu	Zpráva běžícímu agentovi	Vlastní řídicí program	Agent jiné firmy
KDY: nezávislé dílčí úlohy: průzkum modulů, revize, testy vedle	KDY: předávka mezi vlákny, lidmi i nástroji (Codex, Claude Code)	KDY: napsat agentovi, který už běží, jako byste psali vy	KDY: váš skript spouští a řídí víc agentů	KDY: agent jiného týmu nebo dodavatele přes síť
STEJNÁ SESSION	ZAČNĚTE TADY			JINÝ SYSTÉM, JINÝ DODAVATEL
`spawn_agent`	`BRIEF · PROGRESS`	`codex queue`	`codex app-server`	`A2A`
výsledek vrátí jen hlavnímu agentovi; interní nástroje Codexu	nic navíc; trvalé, v Gitu, jde zkontrolovat	interní rozhraní Codexu na vašem počítači, ne standard	totéž interní rozhraní; experimentální, kód na vás	standard; Codex ho sám neumí, síť a přihlášení

Stav patří do souboru. Zpráva jen řekne, kam se podívat.

Dva Codexy nad jedním repem

Implementátor a reviewer, každý ve vlastní pracovní kopii. Předávka přes soubory.

Agent A: implementátor

```
› git worktree add -b feat/42 `
    ..\erp-impl
› cd ..\erp-impl; codex
  /rename impl

› Úloha je v BRIEF.md. Po každém
  kroku zapiš PROGRESS.md
  a commitni.
  ... krok 1 hotov, commit

# vy nebo skript:
› codex queue --thread review `
    --message "Krok 1 hotov,
    viz PROGRESS.md"
```

SDÍLENÉ SOUBORY

BRIEF.md
cíl, rozsah, kdy je hotovo; píše člověk

PROGRESS.md
hotovo, další krok, ověření; píše jen A

REVIEW.json
nálezy podle schématu; zapisuje CLI z B

feat/42
větev v Gitu: commity A vidí B hned

Agent B: reviewer, jen čte

```
› git worktree add --detach `
    ..\erp-review feat/42
› cd ..\erp-review
› git checkout --detach feat/42
  # nejnovější commit od A

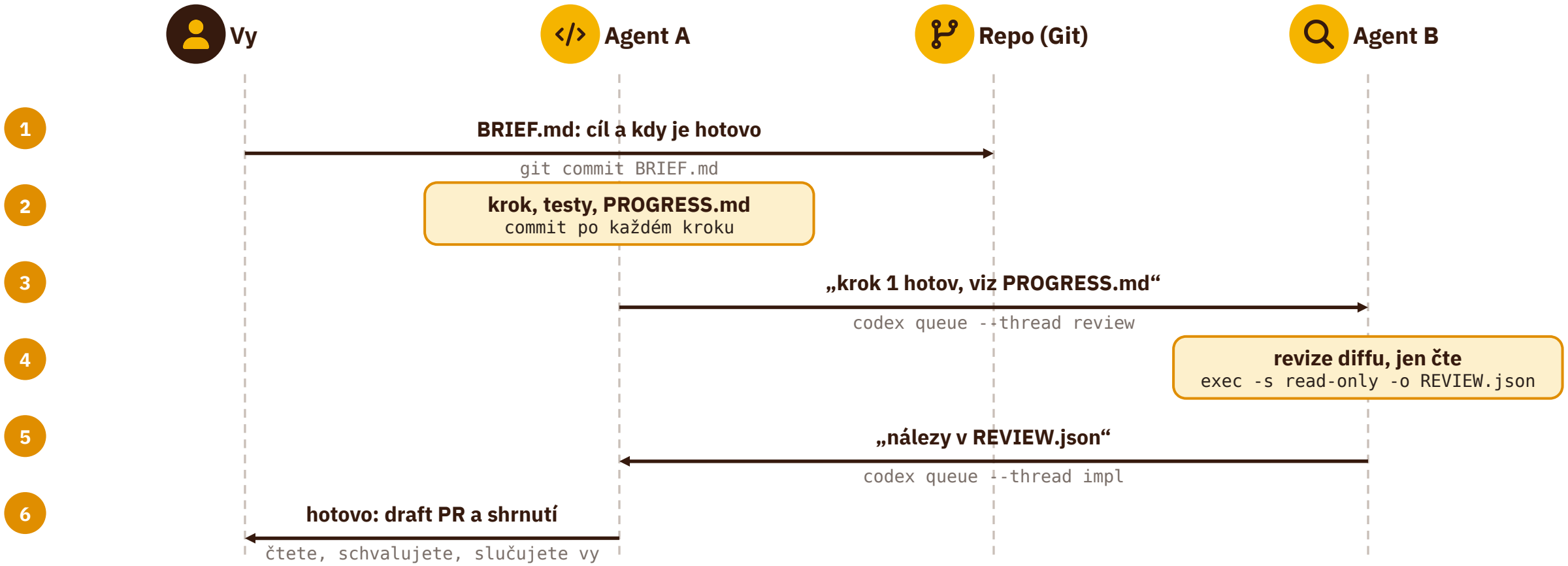
› codex exec -s read-only `
    --output-schema nalezy.json `
    -o ..\erp-impl\REVIEW.json `
    "Revize posledního commitu
     podle Code Review Rules"

› codex queue --thread impl `
    --message "Nálezy v REVIEW.json"
```

Každý soubor má jednoho zapisovatele. Zprávy jen budí, pravda je v Gitu.

Předávka krok za krokem

Protokol bez frameworku: soubor, zpráva, schéma, člověk



Každá zpráva odkazuje na soubor. Kdo přijde pozdě, čte repo, ne historii chatu.

Zásady pro komunikaci agentů

Co se osvědčilo v provozu, i mimo Codex

01

Stav v souboru, ne ve zprávě

Zpráva zmizí s vláknem. Soubor v Gitu přežije restart, kolegu i jiný nástroj.

02

Strukturovaný výstup

--output-schema: JSON, který druhý agent nebo skript přečte bez výkladu.

03

Jeden zapisovatel na soubor

PROGRESS.md píše jen A, REVIEW.json jen B. Jinak se přepisují.

04

Vlastní pracovní kopie

git worktree pro každého agenta. Ve stejné složce si rozbijí build i diff.

05

Zpráva od agenta je cizí vstup

Práva se nepředávají: reviewer jen pro čtení nezačne psát, protože mu to A napsal.

06

Strop kol a rozpočtu

Dva agenti si umí odpovídat donekonečna. Nejvýš tři kola, pak člověk.

07

Ověřit doručení

Agent po tahu stojí a čeká. Hlídejte stav v PROGRESS.md, ticho není „hotovo“.

08

Návody z internetu ověřit

codex mcp-server od verze 0.154 není. Návody na „Codex jako MCP server“ neplatí.

Víc agentů neznamená víc autonomie. Znamená víc předávek a každá potřebuje pravidlo.

Co si odnášíte

1. Co z dneška použijete zítra jako první?
2. Kde vás agent ve vašem kódu nejvíc zbrzdí?
3. Co musí rozhodnout vedení nebo IT, abyste mohli pokračovat?



Tři věci na závěr

ictPRO

1

Setup je kód

Config, AGENTS.md, skilly a hooky v repu, přes PR, s vlastníkem.

2

Jistota před rychlostí

Charakterizace, golden data a paralelní běh dřív, než se změní výpočet.

3

Člověk podepisuje

Očekávané hodnoty, schválené snapshoty, sloučení. Agent nikdy.

V legacy kódu nevyhrává nejrychlejší agent, ale tým s nejlepšími branami.

Otázky a diskuze

jan@lakmoos.com



BONUS PRO ZÁJEMCE

Bonus

Hlubší témata podle zájmu

PODLE ČASU



OWASP Top 10 pro agentní aplikace (2026)

Co z toho se týká coding agenta ve vašem týmu

Riziko	Jak vypadá u vás	Opatření
ASI01 Goal Hijack	instrukce schovaná ve work itemu, komentáři, README závislosti	cizí texty jako data; síť vypnutá
ASI02 Tool Misuse	MCP s právem zápisu „pro jistotu“	jen čtení, enabled_tools
ASI03 Privilege Abuse	agent dědí PAT seniora, connection string v env	vlastní identita, filtry env
ASI04 Supply Chain	npx MCP server bez verze	pinovat, allowlist v requirements
ASI05 Code Execution	agent spouští skripty z repa závislosti	sandbox workspace-write
ASI06 Memory Poisoning	otrávené AGENTS.md nebo skill v PR	review instrukcí jako kódu

Agent s privátními daty, cizím obsahem a cestou ven: uřízněte aspoň jednu nohu.

Jak to dělají velcí: migrace s LLM

Všechny úspěchy mají stejný tvar: úzká změna, strojové vyhledání, build a testy, člověk

Kdo	Co	Jak a čísla (vlastní odhady firem)
Google	int32→int64, JUnit 3→4, Joda→java.time	int32→int64: ~80 % změn od AI; celkově odhad −50 % času; úzké hrdlo je review
Airbnb	3 500 testů Enzyme → RTL	1,5 roku odhad → 6 týdnů; opakování s chybou zpět do promptu
Amazon	Java 8/11 → 17	„4 500 vývojářských let“; recepty + LLM, build jako brána
Meta	TestGen-LLM, ACH	73 % doporučení přijato; člověk vidí jen ověřeně lepší testy
Morgan Stanley	9 mil. řádků COBOL	agent nepřepisuje, ale píše specifikaci starého kódu
Salesforce	migrace balíčku Apex → Java	graf závislostí, pořadí od listů ke kořeni

Vzor Morgan Stanley: agent nejdřív legacy kód vysvětlí a sepíše pravidla, teprve pak se mění.

Lokální model pro dokumentaci

Úloha, kde malý model stačí

Vyhledávání podle významu

- Dokumenty rozdělit na kusy
- Převést na vektory lokálním modelem
- Hledat podobnost místo WHERE … LIKE
- Data neopustí firmu

Kde je limit

- Kvalita odpovědí pod cloudovými modely
- Hardware a provoz na vás
- Vhodné pro hledání, ne pro psaní kódu
- Codex umí lokální model přes --oss

Na vyhledávání v dokumentaci je lokální model dobrá volba. Na agenta zatím ne.

Dva nástroje, jedny instrukce

Když část týmu používá Codex a část Claude Code

Codex

- Čte AGENTS.md
- Skilly v .agents/skills
- Role v .codex/agents

Claude Code

- Čte CLAUDE.md
- CLAUDE.md může jen odkázat: @AGENTS.md
- Vlastní skilly a sub-agenti

Pravidla pište jednou do AGENTS.md. Ostatní nástroje na něj odkážou.

Codex ↔ Claude Code: mapování pojmů

Pro toho, kdo zkouší oba. /import v Codexu převezme setup z Claude Code.

Codex	Claude Code
AGENTS.md, AGENTS.override.md	CLAUDE.md (může importovat @AGENTS.md)
~/.codex/config.toml, profily *.config.toml	~/.claude/settings.json, .claude/settings.json
requirements.toml, Agent Security	managed-settings.json
sandbox_mode, approval_policy, rules	permissions allow / deny / ask
hooky (stejné názvy událostí)	hooky v settings.json
.agents/skills/*/SKILL.md	.claude/skills/*/SKILL.md
.codex/agents/*.toml	.claude/agents/*.md
`codex exec --json --output-schema`	claude -p --output-format json --json-schema

Instrukce, skilly a testy v repu jsou přenositelné. Oprávnění a config se přepisují.

ExecPlan: plán, který přežije hodiny práce

Postup OpenAI pro dlouhé úlohy (PLANS.md)

Kostra ExecPlanu

```
# <Cíl jednou větou>
## Purpose / Big Picture
## Progress (checkboxy, časy)
## Surprises & Discoveries
## Decision Log
## Outcomes & Retrospective
## Context and Orientation
## Plan of Work
## Concrete Steps
## Validation and Acceptance
## Idempotence and Recovery
## Artifacts and Notes
## Interfaces and Dependencies
```

Proč to funguje

```
- soběstačný: nové vlákno nebo
  kolega naváže bez historie
- „živý“: agent ho aktualizuje
  po každém kroku
- Validation = příkazy a
  očekávaný výstup, ne pocit
- Recovery = jak se vrátit,
  když krok selže

› Přečti PLANS.md a vytvoř
  ExecPlan pro <úlohu>. Nic
  neimplementuj, ukaž mi ho.
```

U úlohy na víc než jedno vlákno je plán v souboru povinný. Kontext vlákna se ztratí, soubor ne.

Časté chyby při zavádění v týmu

Viděno jinde, ať je nemusíte opakovat

1	**Každý po svém**	Pět lidí, pět sad instrukcí	*AGENTS.md v repozitáři*
2	**Metrika objemu**	Chválí se řádky, ne výsledky	*doba průchodu, přepracování*
3	**Bez vlastníka**	Instrukce zastarají za měsíc	*jméno u každé věci*
4	**Všechno najednou**	Agenti na všem, kontrola na ničem	*jedna úloha, pak další*
5	**Výchozí nastavení**	Nový model spálí rozpočet	*config.toml týmu*

Většina problémů je organizační. Technika je ta snazší část.